



School of Physics, Engineering and Computer Science

MSc Computer Science

Artificial Intelligence and Robotics

Module: MSc CS Project

Final Project Report

FPGA-Based Driver Distraction Detection System Using Image Classification



Name : Dulhara Anjana Langappuli
Student ID : 22044421
Level : Level 7
Academic Year : 2025-26
Supervisor : Professor Saeid Gorgin

I confirm that this report has been critically proof-read and quality checked by me, and that it is free from grammar, spelling and formatting errors to the best of my knowledge, meeting the standards of clarity, coherence and presentation expected of this submission.

Declaration

This report is submitted in partial fulfilment of the requirement for the degree of Master of Science in Computer Science (Artificial Intelligence and Robotics), at the University of Hertfordshire.

I hereby declare that the work presented in this project and report is entirely my own, except where explicitly stated otherwise. All sources of information and ideas, whether quoted directly or paraphrased, have been properly referenced in accordance with academic standards. I understand that any failure to properly acknowledge the work of others could constitute plagiarism and may result in academic penalties.

I did not use human participants in my MSc Project.

I hereby **give** permission for the report to be made available on the university website provided the source is acknowledged.

Name: Dulhara Anjana Langappuli

Student ID: 22044421

Date: 28th August 2026

Contents

Abstract	1
1 Introduction	2
1.1 Problem overview	2
1.2 Current issues and the gap this project addresses	2
1.3 Project details	3
1.4 Aims and objectives	3
1.5 Research questions	3
1.6 Feasibility, commercial context and risk	4
1.7 Report structure	4
2 Literature Review	5
2.1 How the literature for this project was selected	5
2.2 Image-based driver distraction detection	7
2.3 Subject-wise evaluation, dataset bias and domain shift	7
2.4 Multi-view and cross-view driver monitoring	7
2.5 Low-light, infrared and near-infrared driver monitoring	8
2.6 Quantisation and embedded/FPGA deployment	8
2.7 Gaps identified and positioning of this project	9
3 Methodology	11
3.1 Project management approach	11
3.2 Dataset preparation and splitting strategy	13
3.2.1 Single-view baseline: State Farm	13
3.2.2 Multi-view extension: SAM-DD	13
3.2.3 Lighting extension: the Low-Light Driver Distraction Dataset	14
3.2.4 Subject-wise integrity across every dataset and stage	14
3.3 Model selection and training	15
3.4 Quantisation and on-device deployment	15
3.4.1 Model conversion	15
3.4.2 Board bring-up and the ARM32 runtime issue	16
3.5 FPGA alert-controller design and verification	16
3.6 Alert delivery, tools and test strategy	16
3.7 Ethical, legal, professional and social issues	17

4	Quality and Results	19
4.1	Single-view baseline: model comparison (RQ1)	19
4.2	Cross-view generalisation (RQ2)	20
4.3	Lighting-domain robustness (RQ3)	23
4.3.1	The day-only baseline collapses	23
4.3.2	Training on real night data recovers most of the gap	23
4.4	Quantisation: accuracy, size and latency (RQ4)	25
4.4.1	Model size	25
4.4.2	Accuracy	25
4.4.3	The consolidated software-to-hardware comparison	26
4.5	On-device deployment on the ZC702 (RQ4)	26
4.5.1	Establishing runtime correctness first	26
4.5.2	Accuracy on real hardware	27
4.6	FPGA alert-controller verification and synthesis (RQ5)	27
4.7	Class-level error analysis across all three stages	29
4.8	Summary against the research questions	29
5	Evaluation and Conclusion	31
5.1	Final evaluation against objectives	31
5.2	Project management	31
5.3	Insights gained	32
5.4	Reflection on challenges	32
5.5	Comparison to the literature	33
5.6	Limitations	33
5.7	Commercial context and future work	34
5.8	Conclusion	34
	References	36
A	Systematic Search and Screening Detail	39
B	Dataset Splits and Composition	41
C	Full Per-Class Classification Reports	43
D	Quantisation Raw Figures	45
E	FPGA Synthesis Report Excerpts	46
F	Alert Delivery Path and Its Verification	47
G	Physical Buzzer Alert Output	49
G.1	The board could not control the header pin at all	49
G.2	The overlay	49
G.3	Root permission and two unrelated Jupyter defects	51
G.4	Driving a passive buzzer	51
G.5	Integration	51

H Repository and Notebook Map	52
I Supporting Documentation	53

List of Figures

2.1	PRISMA-style selection process for the systematic search underpinning this review (90 records identified, 20 core studies selected).	6
2.2	Two patterns from the systematic survey. Accuracy spread reflects differing datasets and split protocols as much as differing method quality, and measurements taken on an embedded target are comparatively rare.	6
2.3	The surveyed research landscape and this project’s pathway through it, produced during the systematic survey that preceded the practical work.	10
2.4	Dataset and method coverage across the core studies. Sparse regions indicate combinations that are under-explored rather than combinations that do not work.	10
3.1	Detailed project schedule, with the literature survey and report writing shown as parallel tracks.	12
3.2	Class distribution across the State Farm subject-wise train/validation/test splits.	13
4.1	Single-view model comparison across accuracy and F1 metrics.	19
4.2	Accuracy against laptop throughput for the three baseline models, showing the trade-off underlying the MobileNetV2 selection.	20
4.3	Cross-view MobileNetV2 training curves for both phases, with the restored best epoch marked.	21
4.4	Cross-view confusion matrices before and after fine-tuning, mixed-view held-out test set.	22
4.5	Qualitative predictions from the fine-tuned cross-view model on held-out test images spanning both camera views.	22
4.6	Low-light MobileNetV2 training curves for both phases. Both ran their full ten epochs, and the best epoch of the fine-tuning phase was the last.	24
4.7	Low-light confusion matrices for the day-only baseline, frozen-backbone and fine-tuned models. The baseline’s collapse into two predicted classes is visible as two dominant columns.	24
4.8	Qualitative predictions from the fine-tuned low-light model on held-out real night-time images.	25
4.9	On-device (ZC702, TFLite INT8) against reference (Keras float32) accuracy on the same 300-image held-out sample, before and after fine-tuning.	27
4.10	Post-synthesis resource utilisation for the alert controller on xc7a35tcpg236-1.	28
B.1	State Farm dataset composition.	42

G.1	The buzzer overlay's Vivado block design. A Zynq7 Processing System block provides AXI/clock/reset plumbing to the already-running processor. A single-bit AXI GPIO block is the only new circuit, exposed externally as buzzer_o on package pin V7.	50
-----	---	----

List of Tables

3.1	Subject-wise splitting applied consistently across all datasets and stages. No subject appears in more than one partition, and all reported test figures come from the held-out subjects listed here. The Low-Light daytime rows are excluded from training for the reason given in Section 3.2.3.	15
4.1	Single-view model comparison on the State Farm subject-wise test set (3,086 images, 4 held-out drivers).	19
4.2	Deployment-oriented measurements on the development laptop (GPU-accelerated). These are laptop figures and are not comparable with the on-device figures in Table 4.6.	20
4.3	Cross-view MobileNetV2 accuracy, frozen-backbone against fine-tuned, on the 17,992-image held-out mixed-view test set.	21
4.4	Lighting-domain robustness. The baseline column is the day-trained model applied to night images with no retraining. All test figures come from held-out drivers. .	23
4.5	Quantisation accuracy cost, measured on different held-out samples. The two loss figures differ because INT8 quantisation error is data-dependent, not because either measurement is unreliable.	26
4.6	Consolidated software, deployment and hardware comparison for the fine-tuned cross-view MobileNetV2 and the FPGA alert controller. Laptop and on-device figures are reported separately and are not interchangeable.	26
4.7	Post-synthesis results for the alert controller, <code>xc7a35tcpg236-1</code> , out-of-context. Raw report excerpts are in Appendix E.	28
4.8	Selected per-class results across stages (P = precision, R = recall). Full tables in Appendix C.	29
A.1	Search sources and initial records identified (before duplicate removal).	39
A.2	PRISMA-style screening stages.	39
B.1	State Farm subject-wise split, by driver identifier.	41
B.2	Composition of the unified multi-view metadata table (170,316 rows).	41
C.1	Per-class F1 for the three single-view baseline models, State Farm test set. . . .	43
C.2	MobileNetV2 full per-class report, State Farm subject-wise test set (accuracy 84.67%).	43
C.3	Cross-view MobileNetV2, frozen-backbone against fine-tuned, mixed-view test set (17,992 images).	44

C.4	Lighting stage: day-only baseline (night-only test set, 1,500 images), frozen-backbone and fine-tuned (merged day+night test set, 4,586 images). The baseline's zero-recall rows are the collapse described in Section 4.3.	44
D.1	Exact model artefact sizes.	45
D.2	Quantisation before/after measurements. All samples are drawn from held-out subject-wise test partitions.	45
E.1	Slice logic utilisation (from <code>utilization.rpt</code>).	46
E.2	Design timing summary (from <code>timing_summary.rpt</code>), clock <code>clk</code> constrained at 100 MHz.	46
F.1	Three-layer verification of the alert-delivery path.	47
H.1	Notebook map: AI pipeline.	52
H.2	FPGA, embedded deployment and alert-delivery map.	52

Abstract

Most published driver-distraction detection systems are trained and tested on images from one fixed camera position, in daylight, with speed usually measured only on a laptop, not real hardware. This project asks what happens when each of those three assumptions is removed, and answers each one with real measurements instead of estimates.

Three CNN architectures, a custom CNN, MobileNetV2 and EfficientNet-B0, were trained on the State Farm Distracted Driver Detection dataset. The training used a subject-wise split that holds entire drivers out of training, which avoids the subject leakage caused by a random image-level split. MobileNetV2 was chosen as the best model, with 84.67% test accuracy and 84.01% macro F1. Two robustness tests were then run on real data. For camera view, State Farm was combined with the SAM-DD multi-view dataset, and a frozen-backbone model reached only 75.94% mixed-view accuracy, uneven across views, and fine-tuning raised this to 91.76%. For lighting, the day-trained model was tested on real night footage recorded under active infrared light, replacing an earlier plan to make synthetic infrared-like images, since a synthetic transform can only approximate real infrared data, not replace it. The model's accuracy collapsed to 19.53%, with zero recall on eight of the ten classes. Training on real night data brought night accuracy back up to 71.13%, at a cost of 3.5 points of daytime accuracy.

The fine-tuned model was quantised to INT8 TensorFlow Lite, reducing its size $8.00\times$, from 20.78 MB to 2.60 MB, for a cost of 2.7 accuracy points. It was then deployed on a Xilinx ZC702 board, after fixing a CPU instruction-set mismatch in the only published runtime for that platform. On-device accuracy was 77.3%, with 100 out of 100 prediction agreement against the reference model. Inference took about 9.15 seconds per image, roughly $150\times$ slower than the laptop figure and the clearest limitation of this deployment. A separate FPGA alert-decision stage was verified in simulation against real model predictions, including real misclassifications, and synthesised to 18 LUTs and 13 flip-flops, with 7.276 ns of positive timing slack.

The results show three things. First, camera-view and lighting robustness have to be trained for on purpose, they do not happen by themselves. Second, the cost of embedded deployment falls almost entirely on the classifier and not on the logic around it. Third, measuring a robustness problem before fixing it is what makes the fix easy to understand.

Chapter 1

Introduction

1.1 Problem overview

Distracted driving is one of the biggest causes of road traffic collisions. Texting, handheld phone use, eating and drinking, adjusting in-cabin controls and talking to passengers all pull a driver’s attention away from the road for long enough to matter at speed. Camera-based monitoring is one of the most direct ways to detect this automatically, and image classification has become the standard approach, where a model learns a fixed set of distraction categories from a single camera and flags it when a driver falls into one.

The practical problem is that almost every published system, including the first phase of this project, is trained and tested under one fixed set of conditions, one camera position and one lighting environment. Such a model has not been shown to work if the camera is mounted somewhere else, or if the vehicle is driven at night, and both of these are normal in a real deployment yet neither is usually tested. A second gap follows on from this. Even a model that classifies correctly is not a usable in-vehicle system on its own, since it needs to run on hardware that could actually sit in a car, a real way of turning a classification into an alert, and real measurements of what that hardware costs, not laptop-only estimates.

1.2 Current issues and the gap this project addresses

Three gaps motivate this project. Chapter 2 checks each one against the reviewed literature.

First, camera-view robustness is rarely tested directly. Where multi-view datasets exist, most published work uses only a single view from them, or tests within one view rather than across views. Left and right distraction classes make this worse, not better, since a front-facing camera mirrors the image compared to a side-mounted one, so a texting-left action recorded from the front looks, in pixels, like texting-right from the side, and pooling views together without correcting for this would quietly corrupt the labels.

Second, lighting robustness is talked about often and measured on real data rarely. Where night performance is looked at all, it is often approximated by adding synthetic darkening or infrared-like colour changes to daylight images, a reasonable approximation of robustness but not a substitute for real low-light footage, since it cannot reproduce what an infrared light actually does to a scene, the loss of colour information, the way skin and fabric reflect near-infrared light and the sensor noise of a camera working at its limit. This project originally planned a synthetic experiment like this and dropped it on purpose once real low-light data became available.

Third, the step from a trained model to a working alert system is usually left as a future direction rather than actually attempted. Reported frames-per-second figures are often laptop numbers, not measurements on the kind of hardware a low-cost deployment would actually use, the accuracy and size cost of quantisation is often left out, and the logic that turns noisy per-frame predictions into a stable alert is rarely specified or tested at all.

1.3 Project details

The project is built in two connected phases that match these gaps.

Phase 1 - AI robustness. A single-view baseline is built first. A custom CNN, MobileNetV2 and EfficientNet-B0 are trained on the State Farm Distracted Driver Detection dataset (10 classes, 22,424 images) using a subject-wise split, and compared on accuracy, macro F1, per-class recall and throughput. Two robustness tests are then run separately on real data. Camera-view robustness is tested by combining State Farm (side view) with SAM-DD, a separate dataset giving paired front and side views. Lighting robustness is tested by combining State Farm with the real night-time part of the Low-Light Driver Distraction Dataset (Saad et al., 2020), recorded under active infrared light, measuring how far the day-only model’s accuracy drops before any fix is applied.

Phase 2 - hardware. The fine-tuned model is quantised to INT8 TensorFlow Lite and deployed on a Xilinx ZC702 board, a Zynq-7000 device chosen because it has a general-purpose ARM Cortex-A9 core that can run a full CNN, unlike a plain FPGA with no processor on it. Real on-device latency and accuracy are measured here, not estimated. A separate decision stage implementing the hysteresis logic needed to stop a flickering alert is designed in Verilog, checked in simulation against real model predictions and synthesised for a low-cost FPGA target, and an alert path from the board to a hosted backend to an Android client closes the loop.

1.4 Aims and objectives

1. To prepare and use suitable driver-distraction image data, including real multi-view and real low-light data, for training and testing a camera-view-robust and lighting-robust classification system.
2. To build and test a prototype running on real, low-cost embedded hardware, with an alert output that reacts to sustained distraction rather than a single momentary prediction and to measure the accuracy, size and latency cost of getting there.

1.5 Research questions

- RQ1** How do a custom CNN, MobileNetV2 and EfficientNet-B0 compare when the test drivers are not present in the training data?
- RQ2** How much does accuracy change when a model is tested on a different, unseen camera view and does fine-tuning on multi-view data close that gap?
- RQ3** How much accuracy does a day-trained model lose on real infrared-illuminated footage and how much can be recovered by training on real night data rather than synthetic approximations?

RQ4 What does INT8 quantisation and embedded deployment cost in accuracy, model size and latency before and after conversion and how do laptop figures differ from real on-device performance?

RQ5 Which part of a complete pipeline, the classifier or the decision logic after it, can actually be built and checked in FPGA hardware in the time available?

Every experiment in Chapter 4 is tied to one of these questions. Section 4.8 gives the answer each one produced.

1.6 Feasibility, commercial context and risk

The scope was set around what could actually be built and measured, not just simulated. A full CNN accelerator in FPGA fabric was set aside early on as unrealistic in the time left, so the ZC702’s Cortex-A9 core runs the classifier instead, with the fabric used only for the decision stage, already flagged as a fallback plan in the Interim Progress Report.

The commercial case here rests on cost, not novelty. Driver monitoring is already required in new vehicle types in several markets, but those systems are usually built in by the manufacturer. The retrofit and fleet market is different, since older vehicles, taxi and delivery fleets and driving schools are served by hardware with a much smaller budget, which is why this project targets parts costing tens of pounds, not thousands. There are risks too, drivers resisting a system that feels like surveillance, false alerts carrying liability risk, and the real distance between a research prototype and a safety-certified product, all revisited in Chapter 5.

The main technical risk that actually happened during the project was a low-level mismatch between the only published TensorFlow Lite runtime for this board and its real CPU instruction set (Section 3.4.2). Fixing it, instead of giving up on on-device inference, became one of the biggest pieces of practical work in this project.

1.7 Report structure

Chapter 2 reviews existing work on distraction detection, cross-view and lighting robustness, and embedded/FPGA deployment. Chapter 3 covers dataset preparation, model training, quantisation, the FPGA and buzzer-overlay design and the test strategy. Chapter 4 presents the results with critical analysis, including a combined software/hardware comparison. Chapter 5 checks the project against its objectives, the literature, project management, limitations and future work.

Chapter 2

Literature Review

2.1 How the literature for this project was selected

This chapter is not just a collection of papers gathered while writing. It is based on a systematic, PRISMA-style search and screening exercise, carried out as foundation work in June 2026 and written up separately as a survey paper. Reporting the selection process matters for the same reason PRISMA exists (Page et al., 2021; PRISMA, 2020), since a hidden search method cannot be judged for coverage or bias, and a reader cannot tell whether a missing topic was searched for and not found, or simply never looked for.

Searches ran across IEEE Xplore, Google Scholar, SpringerLink, MDPI, Elsevier and ScienceDirect sources, arXiv, dataset repositories and citation chasing, using three groups of search phrases matching this project’s three strands, RGB distraction classification, detection and embedded deployment, and lighting robustness. The lighting group used terms such as “night-time driver distraction detection” and “near infrared driver monitoring”, added after supervisor feedback found lighting robustness under-covered relative to its importance.

The selection process is shown in Figure 2.1. The search found 90 records, and after 22 duplicates were removed, 68 were screened by title and abstract, 29 of which were excluded for being out of scope, too general or technically incomplete. Thirty-nine full texts were then checked and eight were excluded, leaving 31 eligible papers, of which 20 were chosen as core studies and 11 were kept as supporting literature. A per-source breakdown is in Appendix A.

The papers cited below are drawn from that eligible set, plus a few later sources found during the project, and where one was already among the 20 core studies that is on purpose, not coincidence. Two things from the survey shape how this chapter reads. First, reported accuracy is hard to compare across papers (Figure 2.2a), since a 99% figure from a random image-level split and an 85% figure from a subject-wise split are not measuring the same thing, and the higher number is often actually the weaker result. Second, deployment claims are far more common than deployment evidence (Figure 2.2b). Most studies stop at reported accuracy, fewer report laptop throughput, and only a small number report measurements on the embedded target itself, which is exactly why Chapter 4 makes a point of keeping laptop figures separate from on-device ones.

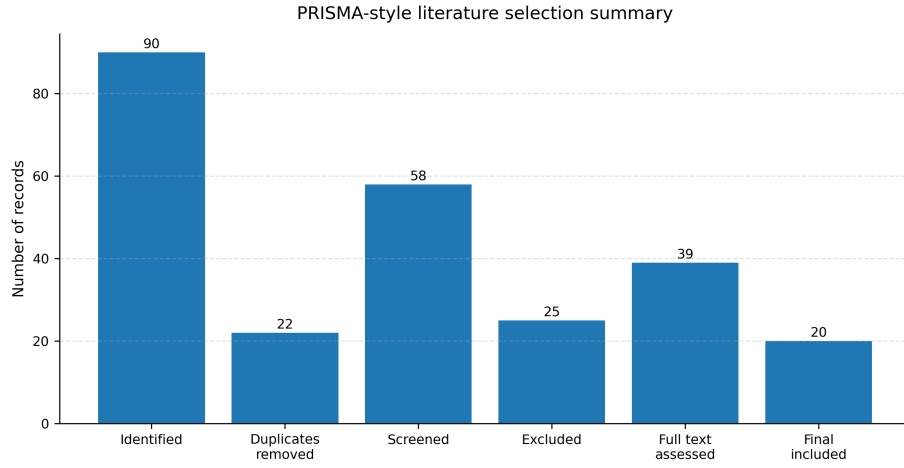


Figure 2.1: PRISMA-style selection process for the systematic search underpinning this review (90 records identified, 20 core studies selected).

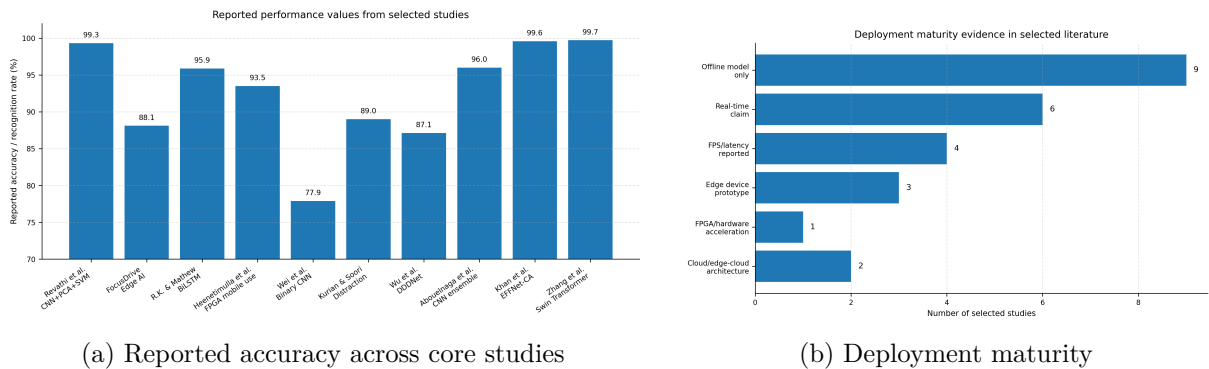


Figure 2.2: Two patterns from the systematic survey. Accuracy spread reflects differing datasets and split protocols as much as differing method quality, and measurements taken on an embedded target are comparatively rare.

2.2 Image-based driver distraction detection

Image-based distraction detection is a well-established use of convolutional neural networks. Valeriano et al. (2018) trained a CNN directly on the State Farm dataset used here and reported strong within-view accuracy, showing that the ten-class taxonomy can be learned from a single side-mounted camera with no extra sensors. AlShalfan and Zakariah (2021) and Aljasim and Kashef (2022) build on this. The former compares several deep architectures on the same problem, and the latter proposes an ensemble that also recommends a response once distraction is detected, treating classification as one stage of a larger decision pipeline, close to the position taken here, where the decision stage after the classifier is its own designed part, not an afterthought.

Abouelnaga et al. (2018) and Eraqi et al. (2019) produced the AUC Distracted Driver dataset and an ensemble method built around it, the clearest proof that the State Farm taxonomy carries over to an independently collected dataset, which is why AUC V2 was the first dataset asked for when the multi-view stage began, although access was not given within the project timeline. Broader reviews by Kashevnik et al. (2021), Horgan et al. (2021) and Dontoh et al. (2025) confirm a pattern more than any single result, that reported systems are almost always tested at one camera position under one lighting condition, treated as fixed dataset properties rather than variables to test.

Two architectures come up again and again in this literature and were used directly here. MobileNetV2 (Sandler et al., 2018) uses inverted residual blocks with linear bottlenecks to keep computation and parameter count low, which is why it is common in driver-monitoring work that expects embedded deployment, both here and in Lu et al. (2025). EfficientNet-B0 (Tan and Le, 2019) instead scales depth, width and resolution together under a fixed compute budget, included here as a comparison point with a different design idea behind it.

2.3 Subject-wise evaluation, dataset bias and domain shift

A methodological problem that recurs across this field, and is treated seriously here, is subject leakage. If the same driver appears in both the training and test partitions, a random image-level split over-estimates real-world accuracy, since the model can learn driver-specific appearance cues, such as clothing, build, seat position and background, instead of the behaviour itself, and consecutive video frames make this even worse. This is the most common reason published figures in this field cannot be compared with each other, and it is why the subject-wise protocol in Section 3.2 is used the same way across every dataset and stage in this project. Wang and Wu (2023) look at a related but different problem, domain shift across recording conditions, and report a measurable drop when a model trained under one set of conditions is tested under another. This points the same way as both robustness findings in this project, although the size of the drop measured here for a real lighting shift (Section 4.3) is far more severe.

2.4 Multi-view and cross-view driver monitoring

The fact that most systems are checked on only one camera view is not because multi-view data or methods do not exist. Jegham et al. (2019) introduced MDAD, a multi-view, multi-modal driver action dataset from synchronised side and front cameras, similar to SAM-DD (Yang et al., 2024) as used here, confirming synchronised-view pairing is an established way to build cross-view

data, not invented for this project. Ortega et al. (2020) and Wang et al. (2023) take this further to larger multi-modal collections. Sengar et al. (2023) take a different approach, pulling pose information per view rather than pooling raw images and combining it with a vision transformer, suggesting viewpoint-aware features are a real alternative to the fine-tuning approach used here.

The most directly relevant work is Bhalla et al. (2025), which frames cross-view monitoring as a two-stage problem, first learning viewpoint-invariant features through contrastive learning across views, then adapting them across sensor types without labelled target data, and reports a large improvement over earlier baselines. That is a much more advanced approach than the one used here, which fine-tunes the last 30 layers of an already-trained backbone on pooled multi-view data. The comparison is useful, since the simpler approach here still recovered most of the accuracy lost when the frozen-backbone model was tested across views (Section 4.2), suggesting well-done fine-tuning on real paired data is a reasonable middle ground for a project of this size, even though contrastive adaptation is still the more principled direction.

Where this project is different from all of the above is that it measures the frozen-backbone cross-view gap on purpose “before” applying any fix. Few of the reviewed papers report an equivalent “before” baseline, which makes it hard to judge how large the raw gap usually is for a MobileNetV2-class model, or how much of each paper’s improvement actually comes from its method rather than from simply having multi-view training data at all.

2.5 Low-light, infrared and near-infrared driver monitoring

Driver monitoring at night is probably the condition where a system like this matters most, and it is clearly under-covered in the RGB-classification literature. The most directly relevant work is Saad et al. (2020), who built the Novel Driver Distraction Dataset with low-lighting support (Hussien et al., 2020), exactly because existing benchmarks were daylight-only, and this is the dataset used in this project (Section 3.2.3). Janveja et al. (2020) treat low-light driver-state monitoring as an embedded systems problem, not just a modelling one.

The most important paper here, in terms of method, is Shen and Lin (2023), who cover daytime and night-time detection using unpaired visible-to-infrared image translation, generating infrared-like images from visible ones. This shows both the strength and the limit of the synthetic approach at once, a real, well-done way to add to scarce night training data, but the paper is clear that translated images only approximate real infrared capture, not replace it. That distinction matters a lot here. The original plan for this project’s lighting stage was exactly this kind of transformation, dropped on purpose once real low-light data became available, since a synthetic transform cannot reproduce what an infrared light actually does to a scene, the loss of colour information, the way skin and fabric reflect near-infrared light and the sensor noise of a camera working at its limit. Because of this, Section 4.3 uses real footage captured under active infrared light, and every conclusion about lighting robustness in this report rests on that real footage, not on an approximation.

2.6 Quantisation and embedded/FPGA deployment

The second half of this project is backed by a strand of literature that cares less about accuracy and more about what a model costs to run once it leaves a workstation. Abushahla et al. (2025) survey quantisation from a hardware point of view, covering both the techniques and the

ARM and RISC-V platforms they target, and confirm that INT8 post-training quantisation, the approach used here, is the standard starting point for edge deployment, usually at a cost of a few percentage points, matching the 2.7 and 4.6-point costs measured on this project’s own models (Section 4.4). Hariharan et al. (2023) report a similar exercise for driver monitoring on edge AI hardware.

Two papers cover the FPGA side directly. Parra et al. (2023) present an open-source method for running a TensorFlow-Lite-quantised CNN through a custom AXI-Lite accelerator on a Zynq board (a Digilent Zybo-Z7, the same SoC family as the ZC702 used here), dealing with the fact that TensorFlow Lite has no built-in way of running on FPGA fabric. Lu et al. (2025) build a dedicated hardware and software accelerator specifically for MobileNetV2-based driver behaviour recognition on a Zynq XC7Z020, the exact chip used on the ZC702, reporting 13.05 FPS at 1.25 W.

Both of these are far more ambitious than the path taken here, which runs the classifier on the Cortex-A9 core using a runtime built from source and only uses the fabric for the decision logic, a choice about scope, not something overlooked. Lu et al. (2025)’s own figures back this up, since their design and verification effort took multiple weeks even with an existing architecture to build on. The cost of the choice made here shows up as speed, roughly 9.15 seconds per inference against those FPS-range figures, reported honestly here as a real limitation, not hidden.

2.7 Gaps identified and positioning of this project

Figure 2.3 places this project against the surveyed landscape. Figure 2.4 shows which method families have been applied to which datasets. The empty regions here are just as informative as the full ones. Four gaps come out of this, and each one maps onto a research question from Section 1.5.

1. **Camera-view robustness is claimed more often than it is measured.** Few papers report the size of the problem for a simple baseline before fixing it (RQ2).
2. **Lighting robustness is usually approximated rather than tested on real infrared-illuminated data** (Shen and Lin, 2023). This project measures the day-to-night collapse on real captured footage (RQ3).
3. **Quantisation and deployment costs are often reported partially or not at all.** Laptop throughput is often given with no matching on-target measurement (RQ4).
4. **The decision stage that turns per-frame predictions into a stable alert is rarely treated as its own designed and tested part of the system** (RQ5).

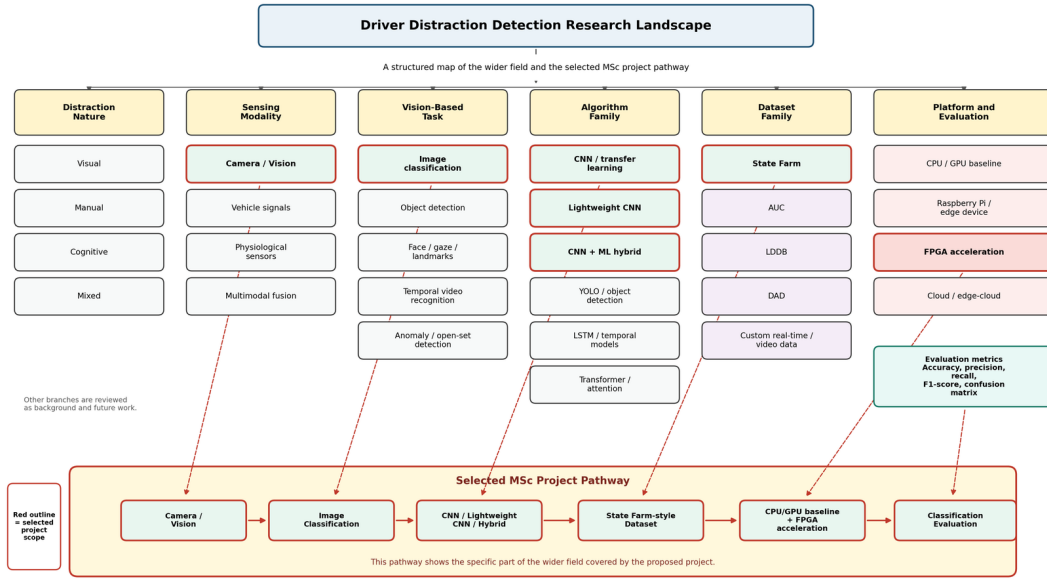


Figure 2.3: The surveyed research landscape and this project’s pathway through it, produced during the systematic survey that preceded the practical work.

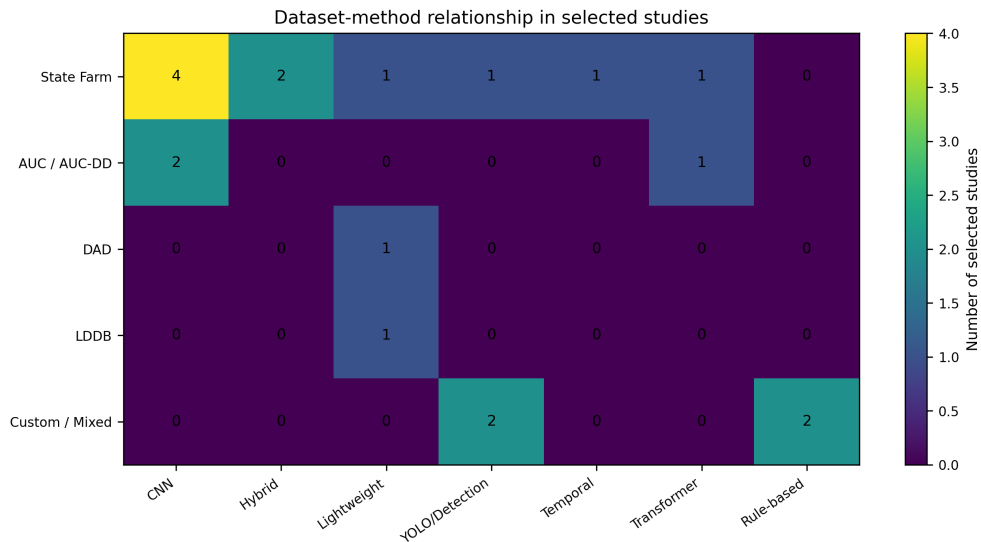


Figure 2.4: Dataset and method coverage across the core studies. Sparse regions indicate combinations that are under-explored rather than combinations that do not work.

Chapter 3

Methodology

3.1 Project management approach

The project ran in two connected phases (Chapter 1). The first was an AI and robustness phase, covering classification, subject-wise evaluation and the two robustness axes. The second was a hardware phase, covering quantisation and embedded and FPGA deployment. Work was tracked notebook by notebook in a version-controlled repository, with progress logs kept next to the code. That discipline produced the audit trail that later caught two real errors, a data-split bug (Section 3.2.3) and a preprocessing mismatch (Section 4.5.1). Supervisor meetings were used to check the direction of the project, most importantly about adding a multi-view stage and about continuing to try on-device inference after an early diagnosis suggested it might not be possible. Figure 3.1 gives a more detailed view of the schedule. Report writing and the literature survey ran alongside the rest of the work rather than as separate steps, so they are shown as overlapping bars.

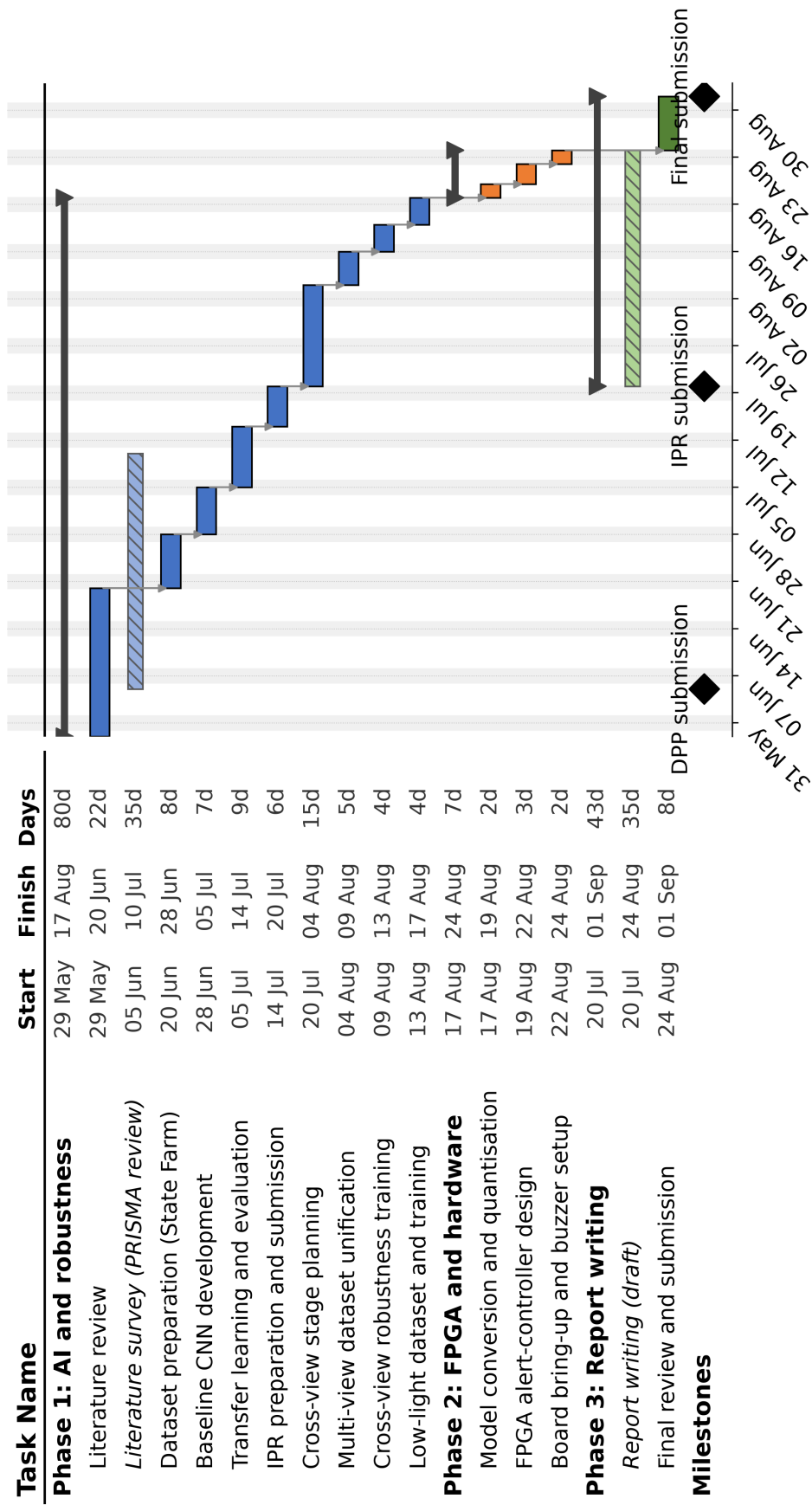


Figure 3.1: Detailed project schedule, with the literature survey and report writing shown as parallel tracks.

3.2 Dataset preparation and splitting strategy

3.2.1 Single-view baseline: State Farm

The State Farm Distracted Driver Detection dataset (State Farm, 2016) gives 22,424 labelled images across ten classes (safe driving, four texting or phone-use classes split by hand, adjusting the radio, drinking, reaching behind, hair or makeup, talking to a passenger), from a single side-mounted camera across 26 drivers. A simple random image-level split and a subject-wise split that holds entire drivers out were compared directly, and the subject-wise protocol was used throughout the rest of this project. A random split lets the same driver’s appearance and background appear on both sides of the split, made worse by near-duplicate video frames landing on both sides too. This inflates the measured accuracy without actually showing that the model generalises to a driver it has not seen. Class balance across the resulting splits was checked, not just assumed (Figure 3.2).

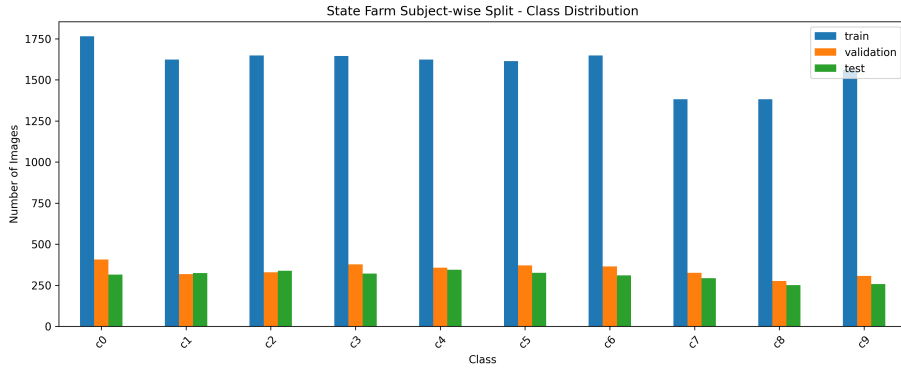


Figure 3.2: Class distribution across the State Farm subject-wise train/validation/test splits.

3.2.2 Multi-view extension: SAM-DD

SAM-DD (Yang et al., 2024) was recorded on an indoor driving simulator across 42 subjects. It was chosen because it gives paired front and side views of the same subjects doing comparable actions, a stronger requirement than just finding a second single-view dataset. This simulator origin is a real limitation, recorded in the metadata rather than glossed over.

SAM-DD has no official file mapping its numbered class folders to class names, so the mapping used here came from a direct visual check of sample images, with a confidence level (high, medium or low) recorded per class. Two SAM-DD classes, adjusting glasses and fatigue or head-drop, do not match anything in State Farm’s taxonomy, and were left out completely rather than forced onto the nearest class, which would have quietly mislabelled the data.

A shared metadata schema was set up (`image_path`, `label_id`, `label_name`, `subject_id`, `dataset_name`, `camera_view`, `vehicle_id`, `lighting`, `modality`, `split`, `is_synthetic`, `notes`) so any dataset could be added without redesigning the pipeline. Left and right classes needed extra care once a second camera angle was added, since a front camera mirrors the image relative to a side-mounted one, so left and right were defined by the driver’s own hand throughout, never by frame side.

3.2.3 Lighting extension: the Low-Light Driver Distraction Dataset

The lighting stage uses the Novel Driver Distraction Dataset with low-lighting support (Saad et al., 2020; Hussien et al., 2020), 70 driver folders across the same ten-class taxonomy, 52,350 images in total, captured through a NoIR-style camera under active infrared light. As explained in Section 2.5, this replaced an earlier plan to make synthetic infrared-like images from State Farm’s daylight data. Preparing it produced three findings, all found by direct checking rather than taken from the dataset’s own documentation.

The dataset is a mix of day and night, not all night. A per-driver brightness check (average greyscale brightness of one frame, cut-off at 40, confirmed by looking directly at over eighteen drivers) found only 9 of the 70 drivers are real night footage. The other 61 were recorded in daylight through the same infrared-capable camera. Taking the dataset’s name at face value would have meant training and testing mostly on daytime images while calling the result night-time. The real night subset is 6,750 images, not 52,350.

Four of the ten classes have an unresolved hand-side problem. `texting_left/texting_right` and `phone_left/phone_right` are each a clear “action” pair (texting held low against phone at the ear), but the driver’s left and right hands could not be told apart from this dataset’s single side-profile angle in any sample checked, confirmed by two separate checks. These classes were kept mapped rather than dropped, matching what was done for SAM-DD, with a low-confidence note recorded that Section 4.7 comes back to.

A real split bug was found and fixed. The first version pooled all 70 drivers together and, purely by chance of the random seed, placed zero of the 9 night drivers in the test split, so a “night” evaluation would have had no night data at all while still looking like it ran correctly. The fix was to split the night and day pools separately, each at 70/15/15. This is exactly the kind of error that gives a confident but wrong result instead of an obvious failure, and it was caught by an explicit check added right after the split.

For the lighting experiment, the merged table combines State Farm (day, RGB, side) with only the night subset on purpose. The 45,600 daytime low-light images were left out too, since putting them in the “day” bucket would bring in a sensor difference alongside the lighting difference, mixing two domain gaps into one number.

3.2.4 Subject-wise integrity across every dataset and stage

Because subject-wise evaluation is the base every number in this report rests on, Table 3.1 states the position for every dataset and stage. The split unit is always the individual driver, no subject appears in more than one partition, and every test figure in Chapter 4 comes from these held-out subjects, never training data, never a validation set reused as a stand-in. Splits were fixed once per dataset and reused unchanged, so the cross-view and lighting experiments are tested on the same held-out State Farm drivers as the baseline. Driver identifiers are in Appendix B.

The unified table holds 170,316 rows. The cross-view experiment draws 80,211 training images and 17,992 test images from it, and the lighting experiment draws 20,399 and 4,586.

Table 3.1: Subject-wise splitting applied consistently across all datasets and stages. No subject appears in more than one partition, and all reported test figures come from the held-out subjects listed here. The Low-Light daytime rows are excluded from training for the reason given in Section 3.2.3.

Dataset	Used in	Split unit	Subjects (tr/val/te)	Images (tr/val/te)
State Farm	all three stages	driver	18 / 4 / 4	15,899 / 3,439 / 3,086
SAM-DD (both views)	cross-view	subject	29 / 6 / 7	64,312 / 16,324 / 14,906
Low-Light DD (night)	low-light	driver	6 / 1 / 2	4,500 / 750 / 1,500
Low-Light DD (day)	excluded	driver	42 / 9 / 10	31,425 / 6,675 / 7,500

3.3 Model selection and training

Three architectures were compared on the State Farm baseline, a custom CNN as a from-scratch reference point, MobileNetV2 (Sandler et al., 2018) and EfficientNet-B0 (Tan and Le, 2019), the last two using transfer learning from ImageNet weights with a new head. All three used the same subject-wise split and were tested on accuracy, macro F1, weighted F1, per-class precision/recall, a confusion matrix, plus throughput and model size as deployment metrics distinct from accuracy. MobileNetV2 was carried forward because of its balance of accuracy and throughput and its comparatively small size.

Both robustness stages reuse the same two-stage protocol so their results can be compared directly. The backbone is first frozen with only the new head trained for 10 epochs, then the last 30 MobileNetV2 layers are unfrozen and fine-tuned at 1×10^{-5} , the recipe already proven in the single-view phase, for a further 10 epochs, with early stopping (patience 5) restoring the best validation epoch. Fine-tuning was not applied from the first epoch on purpose, since updating a large pretrained backbone before the head has learned anything useful risks destroying the pretrained features. Reporting the frozen result next to the fine-tuned one shows how big the underlying robustness problem actually is. Training used a fixed random seed on a single machine, which makes each run reproducible but does not show run-to-run variance (Section 5.6).

3.4 Quantisation and on-device deployment

3.4.1 Model conversion

The trained Keras model (HDF5, `keras_version` 2.10.0 in its own metadata) had to be converted to TensorFlow Lite for the ZC702’s ARM core, in an isolated Python 3.9/TensorFlow 2.10.0 environment matching the model’s exact save version. The main environment (TensorFlow 2.18 and Keras 3) could load the file through a compatibility shim, but the export it produced was one the board’s runtime could not run at all.

A second bug showed up during conversion. The model’s data-augmentation layer kept leaking unsupported operations into the exported graph, even with `training=False` set at trace time, because `Sequential.layers` is a read-only computed property in this Keras version, so clearing it silently does nothing, confirmed by printing the layer list before and after and finding it identical. The fix was to monkey-patch the layer’s bound `.call` method to the identity function before tracing, checked two ways, zero occurrences of the leaked operation names in the exported binary, and outputs numerically identical to the original model. INT8 post-training quantisation was then applied using a 120-image representative dataset (Google, 2026).

3.4.2 Board bring-up and the ARM32 runtime issue

The ZC702 (Zynq-7000, XC7Z020, dual-core Cortex-A9), loaned from the university electronics lab, was set up with the community PYNQ image (AMD/Xilinx, 2026) (Ubuntu 18.04, Python 3.6), needed because the official Vitis-AI and DPU tooling does not support this chip family at all. Bring-up found and fixed two real defects in the image’s own Jupyter service. But the bigger problem was that the only `tflite_runtime` wheel ever published for this platform (2.5.0, the only one that has ever existed) loaded models correctly but crashed with `Illegal instruction` the moment `interpreter.invoke()` started computing. Memory exhaustion was ruled out, both quantisation schemes crashed the same way, and disabling XNNPACK changed nothing. Checking `/proc/cpuinfo` showed this Cortex-A9 supports `vfpv3` but not `vfpv4`, so it has no fused multiply-add instruction, while the published wheel targets `neon-vfpv4`, the profile common to Raspberry Pi 3 and 4 hardware. This is a real CPU instruction-set mismatch, not a configuration or model problem, documented on this SoC family.

The runtime was rebuilt from source directly on the board, which needed CMake to be bootstrapped from source first, because the board’s operating system is older than any usable prebuilt CMake for this architecture. Building natively, instead of cross-compiling, let GCC’s `-march=native` flag detect the actual CPU present, so the compiler never emits an instruction the chip lacks, a more accurate fix than just “targeting VFPv3”. This fixed the crash, confirmed by measuring prediction agreement against the reference runtime (Section 4.5.1).

3.5 FPGA alert-controller design and verification

A full CNN accelerator in fabric was assessed early on, in line with the IPR’s own fallback plan, and judged unlikely to reach a demonstrable standard in the time available (Lu et al., 2025; Parra et al., 2023). Instead, the FPGA work targets the decision stage that comes after the classifier. `distraction_alert_controller.v` implements confidence-hysteresis logic in fixed-point Verilog. It raises an alert only after eight confident “distracted” ticks in a row, and clears it only after eight confident “safe” ticks in a row, using the same 40% threshold as the software-side smoothing fix already proven on the live demo. This mirrors a software fix already proven in the project rather than a separate hardware rule, aiming for a design justified with evidence, not just specified on paper.

Two levels of verification were used. The first was a standalone self-checking testbench that runs the hysteresis logic against synthetic edge cases, ten checks in total. The second replayed 28 real predictions from the trained cross-view model on real test images, sent bit by bit over a simulated UART link through the receive-decode-decide-alert chain, including two real misclassifications, the input the hysteresis logic exists to absorb. The controller was then synthesised out-of-context for a Digilent Basys3 board (`xc7a35tcpg236-1`, Artix-7), a low-cost target chosen on purpose, using Vivado 2017.4 in batch mode.

3.6 Alert delivery, tools and test strategy

To close the loop from a classification to a notification a driver or fleet operator would actually receive, the board runs a continuous capture-classify loop whose hysteresis logic is a tick-for-tick Python copy of the Verilog controller. It posts each alert, with its triggering frame, to a hosted

FastAPI backend that serves an Android client. A physical buzzer wired to the board gives a second, separate alert channel, fired on the same rising edge as the network post. Driving it needed a purpose-built PYNQ overlay, a minimal Vivado block design (Zynq processing system plus one AXI GPIO bit), synthesised, implemented and loaded onto the ZC702’s `xc7z020` silicon. It exposes a pin that was previously not controllable at all, confirmed against the board’s user guide (Xilinx, 2019), the one part of this project’s FPGA work that was verified on real hardware rather than estimated (Section 4.6). Full detail is in Appendix G. The full path was checked in three layers, set out in Appendix F. What was not done is an unattended physical demonstration of someone staying distracted long enough to trigger the alert on their own, and the layered verification here is the honest substitute for that.

The AI pipeline used Python, TensorFlow/Keras (2.10.1 for GPU training on an NVIDIA RTX 3060, 2.10.0 for conversion), OpenCV, NumPy and pandas in Jupyter notebooks with reusable `src/` modules, and hardware work used Vivado 2017.4 and the ZC702’s own PYNQ Linux environment. Testing was applied at every stage, not only at the end. Dataset adapters were checked with explicit assertions, no unmapped labels, no row-count loss, no subject in more than one split, right after each dataset was folded into the unified table, so a data bug would show up before it could waste a training run, which is exactly what caught the night-driver split bug. Model evaluation used held-out subject-wise test sets throughout. The conversion pipeline was checked structurally (no leaked operations in the exported graph) and numerically (matching output against the original model), not just by confirming conversion finished, and on-device deployment was held to the same standard, measuring board-versus-reference agreement directly rather than treating “the crash is gone” as proof of correctness.

3.7 Ethical, legal, professional and social issues

Ethics approval was not needed. All data comes from existing public secondary datasets (State Farm, 2016; Yang et al., 2024; Hussien et al., 2020), with no participants recruited and no third-party data collected, and live qualitative testing used the researcher’s own footage. No dataset is included in the repository, all stored locally and left out through `.gitignore`, matching each dataset’s own terms. SAM-DD in particular is licensed strictly for non-commercial use, with no redistribution allowed.

None of the three datasets records demographic information such as age, ethnicity or body type, so no claim can be made that these figures generalise beyond these specific groups of subjects. A driver-monitoring system also raises a real privacy concern, pointing a camera at a person on every journey, though the design helps somewhat, since inference runs entirely on the board and only a single alert event leaves the device, not a continuous stream. But the alert-image feature does send the one triggering frame, a trade-off that should be user-configurable in any real product. In line with the BCS Code of Conduct (BCS, The Chartered Institute for IT, 2026), this system is described throughout as a research prototype that shows technical feasibility, not a validated safety product. A missed detection and a false alert both carry real consequences in a vehicle, and the second is arguably worse, since a system that cries wolf too often gets ignored or turned off, exactly why the hysteresis stage exists.

In practice, the biggest constraint was getting hold of hardware, not modelling difficulty. The ZC702 was loaned partway through the project, which fixed the earliest possible start date for on-device work, and as a roughly £1,200 asset that had to be returned in working condition, it

ruled out any changes carrying real risk (Section 5.4).

Chapter 4

Quality and Results

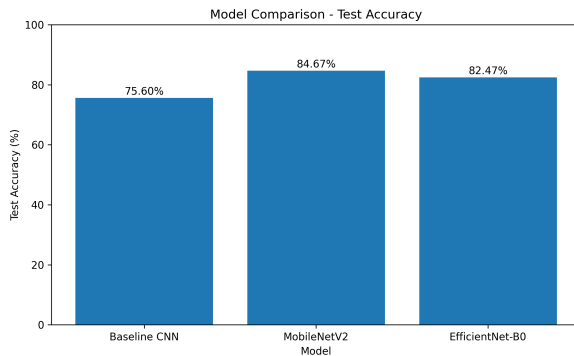
Every result in this chapter comes from the fixed held-out subjects listed in Table 3.1. Full per-class precision, recall and F1 tables for all six trained models are in Appendix C.

4.1 Single-view baseline: model comparison (RQ1)

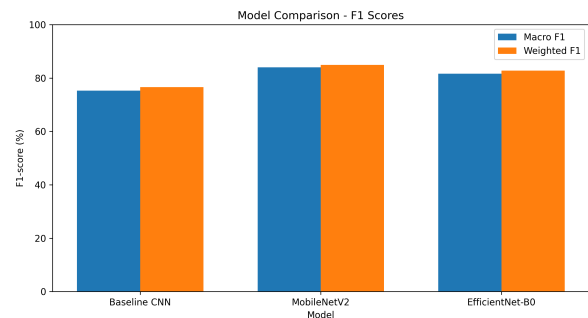
Table 4.1 shows the three architectures trained on State Farm under the subject-wise split. MobileNetV2 got the highest test accuracy (84.67%) and macro F1 (84.01%), ahead of EfficientNet-B0 (82.47% / 81.58%) and the custom CNN (75.60% / 75.27%). EfficientNet-B0’s validation-to-test gap (72.23% against 82.47%) is worth noting, since by chance this validation split was harder than the test split for that model, and a single number should not be over-read under a protocol with only four drivers per partition, one reason Section 5.6 treats the lack of multi-seed data as a real limitation.

Table 4.1: Single-view model comparison on the State Farm subject-wise test set (3,086 images, 4 held-out drivers).

Model	Val. acc.	Test acc.	Macro F1	Weighted F1	Train time (min)
Baseline CNN	60.89%	75.60%	75.27%	76.57%	11.87
MobileNetV2	82.12%	84.67%	84.01%	84.94%	32.03
EfficientNet-B0	72.23%	82.47%	81.58%	82.79%	37.57



(a) Test accuracy



(b) Macro and weighted F1

Figure 4.1: Single-view model comparison across accuracy and F1 metrics.

Deployment-oriented measurements (Table 4.2, Figure 4.2) were taken on the development laptop, not embedded hardware, and that comparison appears separately in Section 4.4.3. MobileNetV2 was not the fastest model here (16.64 FPS against the custom CNN’s 17.61 FPS), but its nine-point accuracy advantage is why it was carried forward, a clear trade-off between accuracy, throughput and size, not a claim that it wins on every measure.

Table 4.2: Deployment-oriented measurements on the development laptop (GPU-accelerated). These are laptop figures and are not comparable with the on-device figures in Table 4.6.

Model	Size (MB)	Inference (ms)	Throughput (FPS)
Baseline CNN	4.96	56.79	17.61
MobileNetV2	20.78	60.08	16.64
EfficientNet-B0	15.75	64.51	15.50

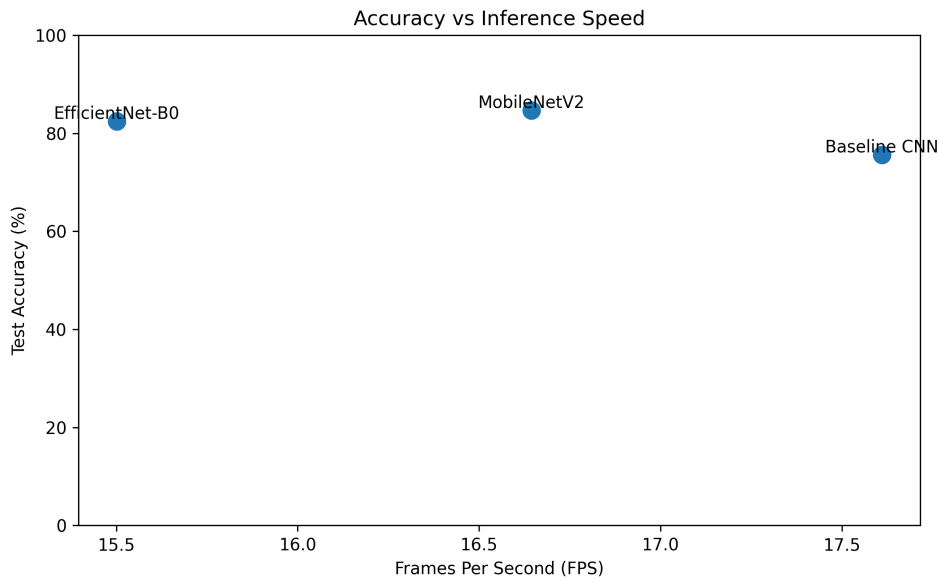


Figure 4.2: Accuracy against laptop throughput for the three baseline models, showing the trade-off underlying the MobileNetV2 selection.

A small qualitative test on out-of-domain photographs taken by the researcher showed domain shift directly. All three models classified a dataset-style image correctly, but across three independently captured photographs only EfficientNet-B0 got two of three actions right, MobileNetV2 got one, and the custom CNN got none. With only three images this is no substitute for the quantitative evaluation that follows, but it was the first sign that strong within-distribution accuracy does not prove robustness to a changed context, and it motivated both robustness stages.

4.2 Cross-view generalisation (RQ2)

MobileNetV2 was retrained on the unified State Farm and SAM-DD table (80,211 training images spanning **side** and **front**) with the backbone frozen. Mixed-view accuracy reached only 75.94%, and it was clearly uneven, 78.79% on **front** against 73.93% on **side**. A model not trained on

purpose to be view-robust is not automatically view-robust, even when both views are present in its training data, and the two are not handled equally well by default.

Unfreezing the last 30 layers and fine-tuning at 1×10^{-5} improved every measure (Table 4.3). Early stopping restored the epoch-5 weights, where validation accuracy peaked at 91.24%, and training accuracy had already gone past 98% by epoch 3, so later epochs were overfitting (Figure 4.3).

Table 4.3: Cross-view MobileNetV2 accuracy, frozen-backbone against fine-tuned, on the 17,992-image held-out mixed-view test set.

Test partition	Frozen-backbone	Fine-tuned	Change
Mixed-view (17,992 images)	75.94%	91.76%	+15.8 pt
side view (10,539 images)	73.93%	91.05%	+17.1 pt
front view (7,453 images)	78.79%	92.77%	+14.0 pt

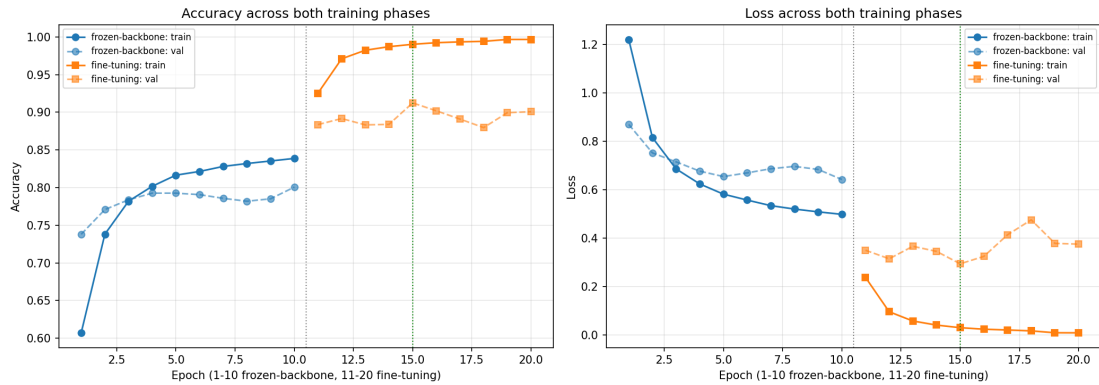


Figure 4.3: Cross-view MobileNetV2 training curves for both phases, with the restored best epoch marked.

All ten classes improved, but unevenly, and this unevenness is actually the more useful result (Section 4.7). Figure 4.5 shows predictions on real held-out images from both views.

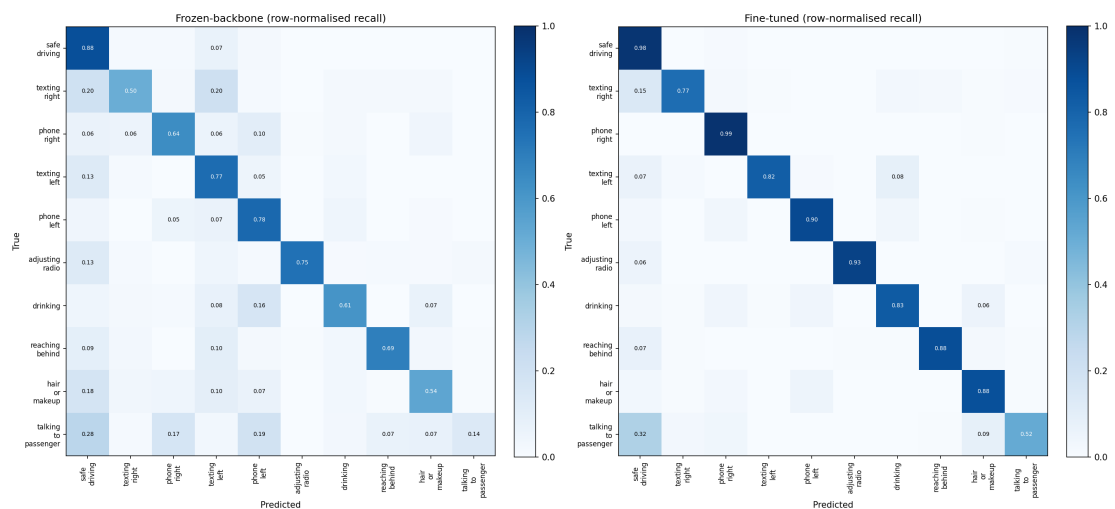


Figure 4.4: Cross-view confusion matrices before and after fine-tuning, mixed-view held-out test set.



Figure 4.5: Qualitative predictions from the fine-tuned cross-view model on held-out test images spanning both camera views.

4.3 Lighting-domain robustness (RQ3)

This stage asks what happens to a day-trained model on real night-time footage captured under active infrared light, not a synthetically darkened daylight image. The results here are the most severe in the whole project.

4.3.1 The day-only baseline collapses

The best single-view State Farm MobileNetV2 checkpoint, the same model that got 84.67% on its own daytime test set, was tested directly on the 1,500 held-out real night images with zero retraining. It scored 19.53%.

That number actually understates how bad the failure is. This is not degradation spread across ten classes, it is a collapse. The model puts almost every night image into just two classes, reaching 99.3% recall on `texting_left` and 96.0% on `reaching_behind`, while recording 0.0% recall on the other eight (Appendix C). It has not just become confused, it has lost the ability to tell classes apart completely, and this is arguably worse than random guessing, since a downstream alert stage would receive a stream of confident, consistent, wrong classifications rather than obvious noise.

4.3.2 Training on real night data recovers most of the gap

A fresh MobileNetV2 was then trained on the merged table, 20,399 training images, State Farm daytime plus only the real night subset, for the reason explained in Section 3.2.3. It used the same protocol as the cross-view stage, so the two robustness results can be compared directly.

Table 4.4: Lighting-domain robustness. The baseline column is the day-trained model applied to night images with no retraining. All test figures come from held-out drivers.

Test partition	Day-only baseline	Frozen-backbone	Fine-tuned
Mixed day+night (4,586 images)	—	62.30%	77.91%
day (3,086 images)	84.67% [†]	64.55%	81.21%
night (1,500 images)	19.53%	57.67%	71.13%

[†]On its own single-domain State Farm test set, shown for reference.

First, training a new classification head on real day and night data, with the backbone still at ImageNet weights, lifts night accuracy from 19.53% to 57.67%, meaning most of the collapse was a classifier-layer problem, not proof ImageNet features are useless on infrared-illuminated images. Second, fine-tuning recovers most of what is left, and night accuracy reaches 71.13%, a gain of 51.6 points over the untreated baseline, done with real captured footage, not a synthetic approximation.

Third, this is a trade-off, not a free win. Even after fine-tuning, day accuracy reaches only 81.21%, 3.5 points below the 84.67% the original single-domain model got on the same test set, since training one shared classifier across both lighting domains costs some single-domain performance to buy cross-domain robustness. This negative transfer is small but real, worth stating plainly rather than describing only the night improvement.

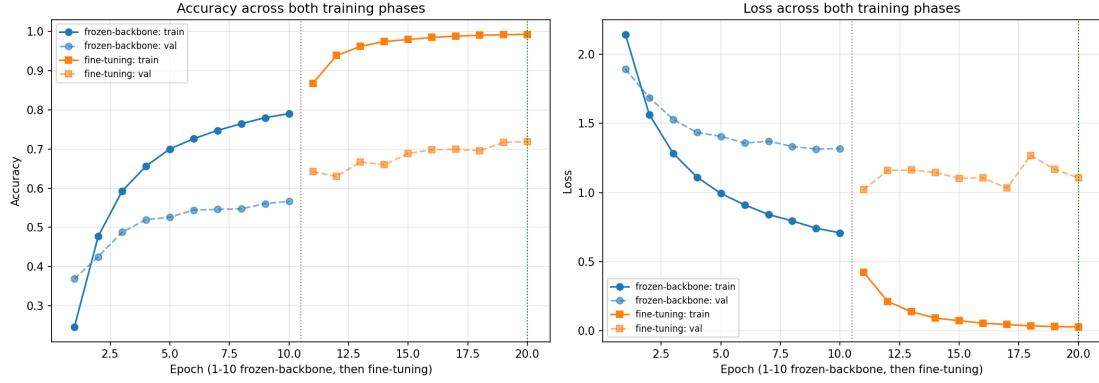


Figure 4.6: Low-light MobileNetV2 training curves for both phases. Both ran their full ten epochs, and the best epoch of the fine-tuning phase was the last.

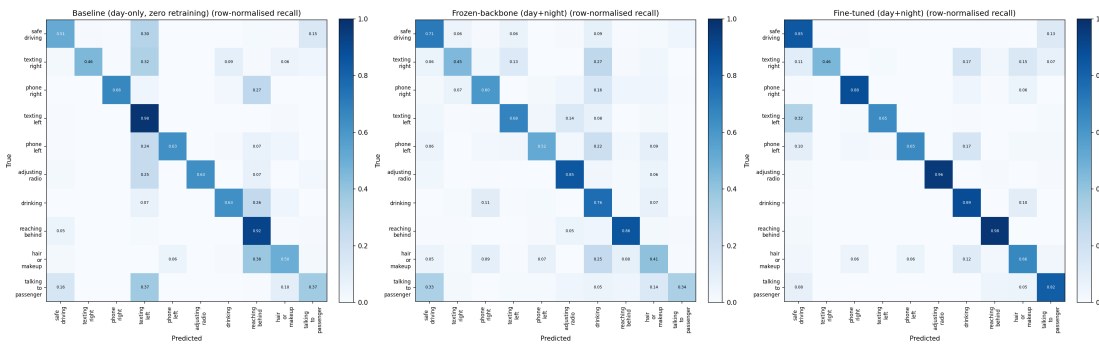


Figure 4.7: Low-light confusion matrices for the day-only baseline, frozen-backbone and fine-tuned models. The baseline's collapse into two predicted classes is visible as two dominant columns.



Figure 4.8: Qualitative predictions from the fine-tuned low-light model on held-out real night-time images.

4.4 Quantisation: accuracy, size and latency (RQ4)

Quantisation is what makes embedded deployment possible at all. Its cost is reported here across all four measures the interim feedback asked for, accuracy, size, latency and throughput, before and after conversion.

4.4.1 Model size

The fine-tuned cross-view Keras model is 21,791,872 bytes (20.78 MB), and the INT8 TensorFlow Lite conversion is 2,723,448 bytes (2.60 MB), a compression ratio of **8.00** $\times$. This is what INT8 quantisation of a mostly float32 model should give, confirming the conversion worked as intended rather than quietly keeping the float weights.

4.4.2 Accuracy

Two before-and-after comparisons were measured on different samples, and both are reported here because the difference between them is informative on its own (Table 4.5).

The first was made at conversion time on a 500-image mixed front-and-side sample, and showed only a 0.8-point cost. The second used a 300-image random draw from the `side` test split, not deliberately balanced, and showed a 4.6-point cost on the same model. These two do not disagree, since INT8 error depends on the data it is measured over, and the 300-image draw is simply harder. The unquantised model itself only scores 59.3% on it, well below the 73.93% `side` average, which is why it was investigated rather than treated as a deployment fault.

Table 4.5: Quantisation accuracy cost, measured on different held-out samples. The two loss figures differ because INT8 quantisation error is data-dependent, not because either measurement is unreliable.

Model	Evaluation sample	Keras float32	TFLite INT8	Cost
Cross-view, frozen	500 mixed-view images	76.80%	76.00%	−0.8 pt
Cross-view, frozen	300 random side images	59.30%	54.70%	−4.6 pt
Cross-view, fine-tuned	300 random side images	80.00%	77.30%	−2.7 pt

The third row matters most, since the fine-tuned model is what was actually deployed. Quantisation costs **2.7 points** on the same 300 images that cost the frozen model 4.6, and the smaller cost on the better-trained model fits with the idea that larger prediction margins are more tolerant of quantisation noise. Both figures sit within the few-point range Abushahla et al. (2025) report as typical.

4.4.3 The consolidated software-to-hardware comparison

Table 4.6 brings the whole pipeline into one place, the laptop float32 reference, the laptop-converted INT8 model, and that same model running on real ZC702 hardware, with the separate FPGA-fabric figures next to them. Keeping laptop and hardware performance clearly apart is the whole point of this table.

Table 4.6: Consolidated software, deployment and hardware comparison for the fine-tuned cross-view MobileNetV2 and the FPGA alert controller. Laptop and on-device figures are reported separately and are not interchangeable.

Stage	Platform	Model size	Accuracy ^a	Latency / throughput
Reference (float32)	Laptop (RTX 3060)	20.78 MB	80.0%	60.08 ms / 16.64 FPS ^b
Quantised (INT8)	Laptop (converted)	2.60 MB	77.3% ^c	not separately timed
Deployed (INT8)	ZC702 (Cortex-A9)	2.60 MB	77.3%	9,150 ms / 0.11 FPS

FPGA fabric, alert-decision stage only, not the CNN (post-synthesis, `xc7a35tcpg236-1`):
18 LUTs (0.09%) · 13 FFs (0.03%) · 0 BRAM · 0 DSP · +7.276 ns slack at 100 MHz

^aSame 300-image held-out **side** sample throughout, so the rows are directly comparable. ^bLaptop latency was measured for this architecture during the baseline evaluation. The fine-tuned model is the same architecture and input size, so the figure carries over, but it was not separately re-timed. ^cThe laptop-converted and on-device INT8 models agreed on 100 of 100 predictions, so they share an accuracy figure by measurement, not assumption.

Quantisation buys an $8\times$ size reduction for a cost of 2.7 accuracy points. Moving from laptop to the embedded target costs almost nothing in accuracy, since the INT8 model behaves the same on both, but it costs roughly **150× in speed**. The fabric alert-decision stage is meanwhile almost free, so the pipeline’s whole cost sits in the CNN, exactly the reason a dedicated CNN accelerator would make sense as the next engineering step (Section 5.7).

4.5 On-device deployment on the ZC702 (RQ4)

4.5.1 Establishing runtime correctness first

Before any on-device accuracy figure could be trusted, the from-source runtime (Section 3.4.2) had to be shown numerically correct, not just crash-free. An early board-versus-laptop com-

parison showed only 71% agreement, which looked like a broken build but was not, since the cause was the test script, which resized images with `PIL.Image.resize()` while training had used `tf.image.resize()`. The deployment script itself was unaffected, since it already used `cv2.resize`, matching `tf.image.resize` 95% of the time. Once preprocessing was matched, board-versus-reference agreement was **100 out of 100**, every prediction identical, which shows the rebuilt ARM32 runtime is numerically faithful to the official implementation, a stronger claim than just saying it does not crash.

4.5.2 Accuracy on real hardware

A 300-image random sample was then evaluated four ways, Keras float32 and on-device TFLite INT8, for both the frozen and fine-tuned checkpoints (Figure 4.9). Fine-tuning improved on-device accuracy from 54.7% to **77.3%**, a gain of 22.6 points measured on real hardware, not just assumed from lab results, which shows the lab improvement survives quantisation and deployment fully intact, not something that could be assumed in advance.

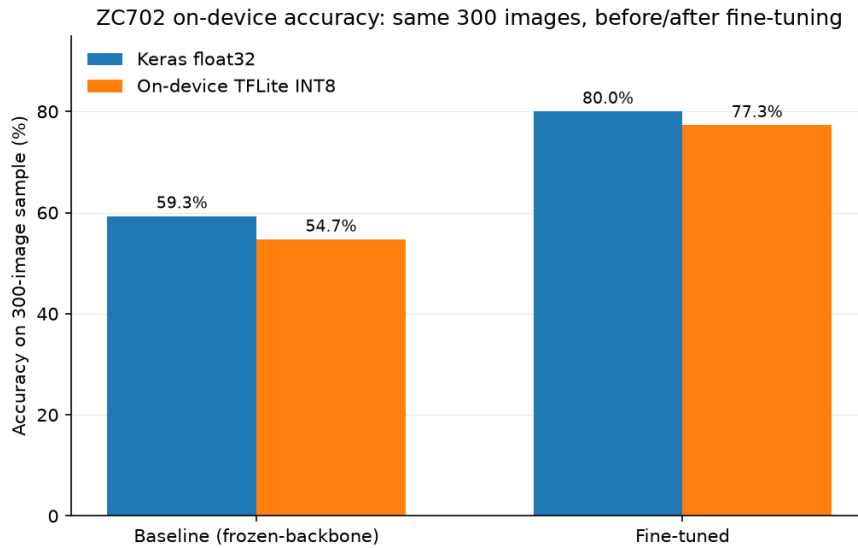


Figure 4.9: On-device (ZC702, TFLite INT8) against reference (Keras float32) accuracy on the same 300-image held-out sample, before and after fine-tuning.

On-device inference took about 9.15 seconds per image, with XNNPACK acceleration turned off on purpose, the known-safe setting this ARM32 target’s crash fix was verified under, and the clearest limitation of this deployment. Section 5.4 explains why re-enabling XNNPACK was looked into but not attempted.

4.6 FPGA alert-controller verification and synthesis (RQ5)

Both testbenches passed in full. The standalone hysteresis testbench passed all ten checks, and the end-to-end testbench replayed 28 real classification outputs over a simulated UART link, including two real model misclassifications, both correctly absorbed without setting off a false alert, exactly the failure mode the design exists to prevent, tested against real wrong model output, not synthetic noise. Out-of-context synthesis for the Artix-7 target (Table 4.7,

Figure 4.10) shows the controller uses a tiny fraction of the device and meets its 100 MHz timing constraint with 7.276 ns of slack to spare.

Table 4.7: Post-synthesis results for the alert controller, `xc7a35tcpg236-1`, out-of-context. Raw report excerpts are in Appendix E.

Metric	Value
Slice LUTs	18 (0.09% of 20,800)
Slice registers (flip-flops)	13 (0.03% of 41,600)
Block RAM	0
DSP slices	0
Worst negative slack at 100 MHz	+7.276 ns
Worst hold slack	+0.272 ns
Failing timing endpoints	0 of 17
Implied theoretical maximum frequency	~367 MHz
Estimated total on-chip power	0.069 W (0.068 W static)

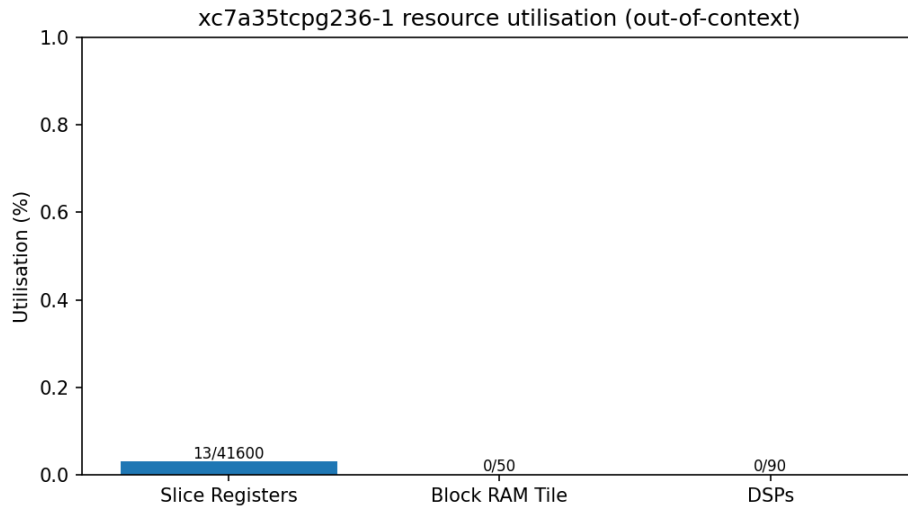


Figure 4.10: Post-synthesis resource utilisation for the alert controller on `xc7a35tcpg236-1`.

First, these are post-synthesis estimates, not measurements from a design actually placed and routed on physical silicon, since this project never had access to a real Artix-7 board, and the target was chosen as a typical low-cost device. Vivado’s own report says final LUT counts after physical optimisation are usually lower, so if anything this figure is conservative, but it is still an estimate. Second, the power figure is a vector-less estimate at the tool’s own medium confidence level. Third, and this matters most, these numbers describe the decision stage only, not the CNN, since the classifier runs in software on the ARM core and is not part of this synthesised design, so nothing in Table 4.7 should be read as the resource cost of running MobileNetV2 in fabric.

What the FPGA work does prove is narrower, but real. The logic that turns a noisy prediction stream into a stable alert is a lightweight, low-power part that fits on the cheapest FPGA devices, verified against real model output including real errors, and reproducible from the scripts submitted with this report.

4.7 Class-level error analysis across all three stages

Reporting only the headline accuracy would hide the more useful structure in these results. Table 4.8 tracks the most diagnostic classes.

Table 4.8: Selected per-class results across stages (P = precision, R = recall). Full tables in Appendix C.

Class	Cross-view frozen		Cross-view fine-tuned		Low-light fine-tuned	
	P	R	P	R	P	R
talking_to_passenger	0.61	0.14	0.60	0.52	0.72	0.82
texting_left	0.42	0.77	0.97	0.82	0.97	0.65
texting_right	0.69	0.50	0.92	0.77	0.99	0.46
hair_or_makeup	0.70	0.54	0.87	0.88	0.60	0.66
drinking	0.75	0.61	0.84	0.83	0.64	0.89
safe_driving	0.89	0.88	0.94	0.98	0.55	0.85

`hair_or_makeup` is the hardest class in the project, not tied to one robustness condition, weakest in the single-view baseline (F1 0.71), weakest again in the low-light frozen model (F1 0.42), and only 0.87 in the cross-view fine-tuned model. Raising a hand to the face looks visually close to several other classes, phone-to-ear in particular, and the object being held is small, often hidden, and the first thing lost when image quality drops.

`talking_to_passenger` improves a lot with fine-tuning but starts close to useless, recall rising from 13.6% to 51.9% in the cross-view model. It is also the smallest class in that test set, 258 examples against `safe_driving`’s 9,006, making its recall the least stable figure in the project. Its stronger lighting-stage result (0.82 recall) is on a more balanced test set and should not be read as this class suddenly becoming easy.

`texting_left` in the cross-view frozen model gives the clearest signal here, precision 0.42 against recall 0.77, where high recall with poor precision means the model was using this class as a catch-all for other classes’ predictions. Fine-tuning moved precision to 0.97 while recall stayed about the same, so the improvement came from better class boundaries, not a flat shift across the board, which is what makes the +15.8 point headline mean something rather than just being large.

`texting_right` in the low-light fine-tuned model shows the opposite imbalance, more likely a labelling problem than a modelling one, precision 0.99 against recall 0.46, reliable whenever it predicts the class but missing more than half the true instances, the pattern expected if a chunk of examples were labelled inconsistently, matching the hand-side ambiguity Section 3.2.3 documents for these four classes. This is the most likely explanation, not a proven one, and telling a real weakness apart from label noise would need a re-checked subset, which was not available here.

4.8 Summary against the research questions

RQ1 — Model comparison.

MobileNetV2 outperformed both alternatives under subject-wise evaluation (84.67% accuracy, 84.01% macro F1) while remaining the most deployable on size and throughput together (§4.1).

RQ2 — Cross-view robustness.

A simply pooled multi-view model shows a real, uneven gap (75.94% mixed, **side** 4.9 points behind **front**). Fine-tuning closes most of it, up to 91.76%, and the class-level evidence shows the gain came from better class boundaries, not a flat shift across the board (§4.2, §4.7).

RQ3 — Lighting robustness.

A day-trained model collapses to 19.53% on real infrared-illuminated night footage, with zero recall on eight of ten classes. Real night training recovers this to 71.13%, at a cost of 3.5 points of daytime accuracy (§4.3).

RQ4 — Quantisation and deployment cost.

INT8 conversion reduces the model $8.00\times$ for 2.7 accuracy points, and it is numerically identical on board and laptop. The embedded target costs roughly $150\times$ in speed (§4.4, §4.5).

RQ5 — What is credibly implementable in fabric.

The decision stage, not the classifier. It was verified against real model predictions, including real misclassifications, and synthesised at 18 LUTs and 13 flip-flops with 7.276 ns of positive slack. These are post-synthesis estimates, not measurements on physical silicon (§4.6).

Chapter 5

Evaluation and Conclusion

5.1 Final evaluation against objectives

Both objectives set out in Chapter 1 were met, though not in the shape originally planned.

The first objective, preparing and using real multi-view and real low-light data, was met in full. The multi-view part went as planned. The lighting part did not, since it was planned as synthetic infrared-like augmentation and was replaced on purpose with training on real captured night footage once that data became available. This directly answers the interim supervisor feedback, which warned that synthetic low-light transforms should be described as an approximation of robustness, not a substitute for real infrared data. Rather than relabelling the experiment to satisfy this, it was replaced with one that does not need the caveat at all.

The second objective, a prototype on real, low-cost embedded hardware with an alert output reacting to sustained rather than momentary predictions plus a measured account of what getting there costs, was also met, though the hardware path changed along the way. The original plan treated a full FPGA-fabric CNN accelerator as the main goal, with a software fallback as backup. What was actually built runs the CNN on the ZC702’s ARM core and only uses the fabric for the decision stage, a considered decision about scope, not a fallback settled for reluctantly. A dedicated accelerator is a multi-week job even starting from an existing architecture (Lu et al., 2025), and the IPR had already flagged this trade-off before it came up.

The quantification work answers that feedback most directly, since it asked that quantisation be checked through accuracy, model size, latency and throughput, before and after conversion, with laptop FPS clearly kept apart from hardware performance. Section 4.4 reports all four measures with real numbers, and Table 4.6 gives the single combined comparison also asked for, with laptop and on-device rows kept separate rather than blended into one throughput claim.

5.2 Project management

The timeline set at the start (May to September 2026, single-view work through July, hardware work in August and September) fell short in one way, it did not expect robustness to become two full stages rather than one small check. Both were added after the qualitative domain-shift test in Section 4.1 made the problem real instead of theoretical. Looking back this was the right call, since the two robustness results are arguably the more original part of this project, but it did compress the hardware phase, a direct cause of later decisions, running the CNN on the ARM core instead of building a custom accelerator, and leaving the XNNPACK question unresolved.

The lesson here is that a robustness finding worth investigating properly needs its own budgeted time from the start, not squeezed into an existing timeline once found. A second lesson is about ordering, since the lighting stage was the last big piece of work finished, leaving no room for it to go wrong. It did not, but that was not a margin the plan had earned.

5.3 Insights gained

The most important technical insight was that a crash on unfamiliar embedded hardware is not necessarily proof the approach cannot work, it can just as easily be evidence of a specific, diagnosable, fixable mismatch between a generic published tool and the exact hardware in front of you. Giving up on on-device inference after the first `Illegal instruction` crash was a reasonable call given the information available at the time. Reopening that decision, working out the VFPv3-against-VFPv4 mismatch, and rebuilding the runtime from source on the board best shows the difference between treating an obstacle as a dead end and treating it as a problem with a findable cause.

The second insight is that robustness does not just show up on its own from seeing varied data. The frozen-backbone cross-view model saw both camera views during training and still showed a large, uneven gap, and the frozen-backbone low-light model saw both lighting conditions and still trailed by seven points on night data. Both problems needed the backbone itself to be adapted, and in both cases the real size of the problem would have stayed invisible if only the after-fix number had been measured. Reporting the “before” figure on purpose is what carries most of the analytical value here.

A third insight is that what looks like a model or hardware problem is sometimes a data-pipeline problem in disguise. Both the preprocessing mismatch (Section 4.5.1) and the night-driver split bug were exactly this kind of error, and both would have produced confident, believable, wrong numbers instead of an obvious failure.

5.4 Reflection on challenges

Two more challenges, beyond the ARM32 runtime issue, are worth reflecting on honestly here, rather than just reporting how they were resolved.

The first is the WiFi adapter that was meant to give the board its own connectivity, and it was never made to work. The first idea was a USB power-budget limit, since several peripherals shared one port through unpowered hubs chained together, so it was tested directly. The competing devices were switched off, removing 1 A of demand, and the driver was re-bound with the whole bus available to it. The failure was byte-for-byte identical, the same four register-access timeouts at the same four offsets, real evidence against the power-budget theory, which shifted the diagnosis to a real driver or chipset incompatibility instead. The honest outcome is a disproven guess and a still-unfixed problem, deprioritised since on-device inference needs no network at runtime, with the value in having tested the guess rather than just asserting it.

The second is inference speed. The fix for the original crash, compiling with `-march=native`, should in theory also let XNNPACK’s runtime CPU dispatch avoid the same mismatch, since it uses the same detection library already working correctly for the compiled NEON kernels, but “should in theory” is not the same as “checked and confirmed”. Some XNNPACK ARM32 source files fix FPU flags directly rather than relying on runtime dispatch, and checking this properly

would have needed a multi-hour rebuild that carried real risk to a working installation the demo depended on. Risking a working deployment for a speed gain this close to a deadline was not a good trade, so this is a reasoned decision not to chase the lead, not an unexamined gap.

5.5 Comparison to the literature

Chapter 2 pointed out that the cross-view literature mostly reports post-fix accuracy with no equivalent unfixed baseline. This project’s frozen-against-fine-tuned comparison (75.94% to 91.76%) points the same way as Bhalla et al. (2025) and Sengar et al. (2023), even though both use more advanced methods. What this project adds is the size of the untreated gap on a standard architecture, something those papers do not report, and something needed to judge how much of any published improvement actually comes from the method rather than just from having multi-view data at all.

On lighting, the 19.53% collapse is more extreme than the gradual drop Wang and Wu (2023) report across different recording conditions, and gives a real measured reference point for the concern behind Shen and Lin (2023)’s image-translation work. The problem their synthetic approach exists to reduce is severe enough to justify the effort, and the fact that real night data recovers 51.6 points suggests real data is still the stronger fix where it can be obtained. On hardware, Lu et al. (2025)’s 13.05 FPS for a dedicated accelerator on the same XC7Z020 chip puts this project’s 0.11 FPS in perspective. That gap of roughly two orders of magnitude is the direct cost of the decision not to build one. This project’s hardware contribution was never going to be a faster result, it is a numerically verified one instead.

5.6 Limitations

Run-to-run variance was not measured, since every result comes from a single training run with a fixed random seed, so no confidence intervals can be given and small differences between conditions should not be read as reliably meaningful. The interim feedback specifically asked for uncertainty across runs, and this report cannot give that honestly, since the data simply does not exist, and generating it would have meant re-running each stage several times, roughly two hours per cross-view run alone, which the remaining time did not allow. Stating this gap directly is the right response, rather than making up an interval or quietly ignoring the request. It matters most for the smaller comparisons. The 3.5-point day-accuracy cost and the 2.2-point margin between MobileNetV2 and EfficientNet-B0 both sit in a range where seed variance could plausibly matter, while the 51.6-point night recovery and 15.8-point cross-view improvement are far too large to be explained that way.

Test partitions are small in terms of subjects. Four held-out State Farm drivers and two low-light night drivers are still a small group, however many images they produce, since images from the same driver are not independent samples of anything.

Label quality in two datasets is not perfect, and this is documented rather than hidden. SAM-DD’s class mapping was rebuilt by visually checking images, with a confidence level recorded for each row, and four low-light classes still carry an unresolved hand-side problem. No demographic information exists in any dataset used here, so no fairness claim can be made either way, and SAM-DD is recorded on a simulator, not in a moving vehicle. The FPGA results are post-synthesis estimates covering the alert stage only. Two further datasets, AUC V2 and

100-Driver, were requested but not granted within the project timeline, so cross-view robustness here rests on two sources rather than four.

5.7 Commercial context and future work

The commercial case sketched out in Chapter 1 still holds, with one big qualification. Component costs stay realistic for a retrofit or fleet product, the alert logic takes up under 0.1% of a low-cost FPGA, the quantised model is 2.6 MB, and the classifier runs on ordinary ARM hardware. But 0.11 FPS is not a workable product figure. At roughly nine seconds per inference and eight ticks in a row needed before an alert fires, this setup would take over a minute to respond to sustained distraction, and that gap has to close before any commercial claim is believable. The non-technical risks are, if anything, bigger, since driver monitoring feels like surveillance to a lot of people rather than help, and false alerts create liability risk while training users to ignore the system.

Four directions follow from this. First, inference speed. Turning XNNPACK back on is a clear next step, and beyond that Table 4.6 makes the case for a dedicated CNN accelerator, since the pipeline’s whole cost sits in the classifier. Second, stronger cross-domain methods, using the contrastive viewpoint-invariant approach from Bhalla et al. (2025) on the unified dataset built here, with the same idea applying to the lighting domain too. Third, broadening the data, since the AUC V2 and 100-Driver requests would add two more independent camera sources, and the 45,600 daytime infrared images left out of the lighting experiment would allow a three-way comparison, day-RGB, day-infrared and night-infrared, separating the lighting gap from the sensor-modality gap. Fourth, repeated-seed runs, to supply the variance data this report has to report as missing.

5.8 Conclusion

This project set out to test three connected claims. The first was that a classifier’s accuracy under one camera view says nothing reliable about its accuracy under another. The second was that the same is true, even more so, for lighting. The third was that a complete, hardware-verified detection-and-alert pipeline is actually achievable on genuinely low-cost embedded hardware, not just estimated for it. All three held up.

A MobileNetV2 model trained under subject-wise evaluation reached 84.67% on State Farm’s side view, but showed a real, uneven cross-view gap when simply pooled across views. Fine-tuning on real paired multi-view data closed most of that gap, up to 91.76%. The same model collapsed to 19.53% on real captured infrared-illuminated night footage, a failure severe enough that the original plan to approximate it synthetically would have understated it, and training on real night data recovered this to 71.13%, at a measured cost of 3.5 points of daytime accuracy. Quantisation compressed the deployed model eightfold for 2.7 accuracy points, proved numerically identical on board and laptop, and cost roughly $150\times$ in speed. And a hardware decision stage was verified against real model predictions, including real misclassifications, synthesised at almost no cost on a low-cost FPGA target.

What connects these results is not any single number, it is a methodological position. Robustness is worth measuring before applying the fix, on real data rather than approximations of it, and on the actual hardware the claim is about rather than whatever hardware is convenient.

Each of those choices gave a less flattering figure than the alternative would have, and each gave a more defensible one. For a system whose failures would happen inside a moving vehicle, that seems like the right way round.

References

- Abouelnaga, Y., Eraqi, H. M. and Moustafa, M. N. (2018), ‘Real-time distracted driver posture classification’. arXiv:1706.09498 [cs.CV].
- Abushahla, H. A., Varam, D., Panopio, A. J. N. and AlHajri, M. I. (2025), ‘Neural network quantization for microcontrollers: A comprehensive survey of methods, platforms, and applications’. arXiv:2508.15008 [cs.LG].
URL: <https://arxiv.org/abs/2508.15008>
- Aljasim, M. and Kashef, R. (2022), ‘E2DR: A deep learning ensemble-based driver distraction detection with recommendations model’, *Sensors* **22**(5), 1858.
- AlShalfan, K. A. and Zakariah, M. (2021), ‘Detecting driver distraction using deep-learning approach’, *Computers, Materials & Continua* **68**(1), 689–704.
- AMD/Xilinx (2026), ‘PYNQ: Python productivity for Zynq’.
URL: <http://www.pynq.io/>
- BCS, The Chartered Institute for IT (2026), ‘BCS code of conduct’.
URL: <https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/>
- Bhalla, A., Hellert, C. and Kasneci, E. (2025), ‘Cross-view cross-modal unsupervised domain adaptation for driver monitoring system’. arXiv:2511.12196 [cs.CV].
URL: <https://arxiv.org/abs/2511.12196>
- Dontoh, A., Ivey, S., Sirbaugh, L., Danyo, A. and Aboah, A. (2025), ‘Visual dominance and emerging multimodal approaches in distracted driving detection: A review of machine learning techniques’. arXiv:2505.01973 [cs.CV].
URL: <https://arxiv.org/abs/2505.01973>
- Eraqi, H. M., Abouelnaga, Y., Saad, M. H. and Moustafa, M. N. (2019), ‘Driver distraction identification with an ensemble of convolutional neural networks’, *Journal of Advanced Transportation* **2019**, 4125865.
- Google (2026), ‘TensorFlow lite: ML for mobile and edge devices’.
URL: <https://www.tensorflow.org/lite>
- Hariharan, J., Rama Varior, R. and Karunakaran, S. (2023), ‘Real-time driver monitoring systems on edge AI device’. arXiv:2304.01555 [cs.CV].
URL: <https://arxiv.org/abs/2304.01555>

- Horgan, J., Hughes, C., McDonald, J. and Yogamani, S. (2021), ‘Vision-based driver assistance systems: Survey, taxonomy and advances’. arXiv:2104.12583 [cs.CV].
URL: <https://arxiv.org/abs/2104.12583>
- Hussien, M., Khalil, M. and Abbas, H. (2020), ‘Novel driver distractions dataset with low lighting support’. Version 2.
URL: <https://data.mendeley.com/datasets/ykmr99nrsg/2>
- Janveja, I., Subramanian, S., Masrur, A. and Rahman, T. (2020), InSight: Monitoring the state of the driver in low-light conditions, *in* ‘Proceedings of the 18th Conference on Embedded Networked Sensor Systems’, pp. 1–14.
- Jegham, I., Ben Khalifa, A., Alouani, I. and Mahjoub, M. A. (2019), MDAD: A multimodal and multiview in-vehicle driver action dataset, *in* ‘18th International Conference on Computer Analysis of Images and Patterns (CAIP)’, Salerno, Italy.
- Kashevnik, A., Shchedrin, R., Kaiser, C. and Stocker, A. (2021), ‘Driver distraction detection methods: A literature review and framework’, *IEEE Access* **9**, 60063–60076.
- Lu, A., Fan, X., Tian, J. and Yang, X. (2025), A hardware-software co-design FPGA acceleration platform for driver behavior recognition based on MobileNetV2, *in* ‘2025 IEEE 2nd International Conference on Deep Learning and Computer Vision (DLCV)’.
- Ortega, J. D., Kose, N., Cañas, P., Chao, M.-A., Unnervik, A., Nieto, M., Otaegui, O. and Salgado, L. (2020), ‘DMD: A large-scale multi-modal driver monitoring dataset for attention and alertness analysis’. arXiv:2008.12085 [cs.CV].
URL: <https://arxiv.org/abs/2008.12085>
- Page, M. J., McKenzie, J. E., Bossuyt, P. M., Boutron, I., Hoffmann, T. C., Mulrow, C. D., Shamseer, L., Tetzlaff, J. M., Akl, E. A., Brennan, S. E., Chou, R., Glanville, J., Grimshaw, J. M., Hróbjartsson, A., Lalu, M. M., Li, T., Loder, E. W., Mayo-Wilson, E., McDonald, S., McGuinness, L. A., Stewart, L. A., Thomas, J., Tricco, A. C., Welch, V. A., Whiting, P. and Moher, D. (2021), ‘The PRISMA 2020 statement: An updated guideline for reporting systematic reviews’, *BMJ* **372**, n71.
- Parra, D., Escobar Sanabria, D. and Camargo, C. (2023), ‘A methodology and open-source tools to implement convolutional neural networks quantized with TensorFlow lite on FPGAs’, *Electronics* **12**(20), 4367.
- PRISMA (2020), ‘PRISMA 2020 flow diagram’.
URL: <https://www.prisma-statement.org/prisma-2020-flow-diagram>
- Saad, M. H., Khalil, M. I. and Abbas, H. M. (2020), End-to-end driver distraction recognition using novel low lighting support dataset, *in* ‘2020 15th International Conference on Computer Engineering and Systems (ICCES)’, pp. 1–6.
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. and Chen, L.-C. (2018), MobileNetV2: Inverted residuals and linear bottlenecks, *in* ‘Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition’, pp. 4510–4520.

- Sengar, N., Kumari, I., Lee, J. and Har, D. (2023), ‘PoseViNet: Distracted driver action recognition framework using multi-view pose estimation and vision transformer’, *IEEE Sensors Journal* . arXiv:2312.14577.
URL: <https://arxiv.org/abs/2312.14577>
- Shen, C.-H. and Lin, S.-H. (2023), Driver distraction detection for daytime and nighttime with unpaired visible and infrared image translation, in ‘2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)’.
- State Farm (2016), ‘State farm distracted driver detection’. Kaggle competition dataset.
URL: <https://www.kaggle.com/competitions/state-farm-distracted-driver-detection>
- Tan, M. and Le, Q. V. (2019), EfficientNet: Rethinking model scaling for convolutional neural networks, in ‘Proceedings of the 36th International Conference on Machine Learning’, Vol. 97 of *Proceedings of Machine Learning Research*, pp. 6105–6114.
- Valeriano, L. C., Napoletano, P. and Schettini, R. (2018), Recognition of driver distractions using deep learning, in ‘2018 IEEE 8th International Conference on Consumer Electronics – Berlin (ICCE-Berlin)’.
- Wang, J. and Wu, Z. (2023), ‘Driver distraction detection via multi-scale domain adaptation network’, *IET Intelligent Transport Systems* .
- Wang, Y. et al. (2023), ‘100-driver: A large-scale, diverse dataset for distracted driver classification’.
URL: <https://zenodo.org/records/8335433>
- Xilinx (2019), ‘ZC702 evaluation board for the Zynq-7000 XC7Z020 SoC user guide (UG850 v1.7)’.
URL: <https://www.mouser.com/datasheet/2/903/ug850-zc702-eval-bd-1596187.pdf>
- Yang, H., Liu, H., Hu, Z., Nguyen, A.-T., Guerra, T.-M. and Lv, C. (2024), ‘Quantitative identification of driver distraction: A weakly supervised contrastive learning approach’, *IEEE Transactions on Intelligent Transportation Systems* **25**(2), 2034–2045.

Appendix A

Systematic Search and Screening Detail

Supporting evidence for Section 2.1. The search was carried out in June 2026. Table A.1 gives the per-source breakdown of the 90 records found before duplicates were removed. Table A.2 gives the stage-by-stage screening counts summarised in Figure 2.1.

Table A.1: Search sources and initial records identified (before duplicate removal).

Source	Records	Main focus of results
IEEE Xplore	32	Driver distraction, CNNs, FPGA, low-light NoIR and IR/NIR studies
Google Scholar	24	Citation search, arXiv papers, dataset papers, review studies
SpringerLink	8	Driver monitoring, edge/cloud systems, deep learning
Elsevier / ScienceDirect-related	8	Intelligent systems, driver behaviour, accident-related monitoring
MDPI	7	Sensors, driver state monitoring, multimodal recognition
Dataset repositories, citation tracking	6	Mendeley low-light dataset, DMD, Zenodo dataset records
arXiv and preprint sources	5	DMD, edge AI, synthetic data, emerging methods
Total identified	90	Records before duplicate removal

Table A.2: PRISMA-style screening stages.

Stage	Records
Identified from databases and citation search	90
Duplicates removed	−22
Screened by title and abstract	68
Excluded at title/abstract screening	−29
Full-text papers assessed for eligibility	39
Excluded after full-text assessment	−8
Eligible papers	31
Core studies selected for detailed comparison	20
Retained as supporting literature	11

Records were excluded at title and abstract screening for being outside the scope of camera-based driver monitoring, too general to give technical detail, or technically incomplete. Records were excluded at full-text assessment for lacking a clear dataset, method, evaluation result or stated technical contribution. The inclusion criteria covered studies using RGB, low-light, NoIR, IR, NIR, depth, thermal or multimodal sensing and studies reporting a dataset, camera setup, metrics or split protocol.

Appendix B

Dataset Splits and Composition

Supporting evidence for Section 3.2.4 and Table 3.1. Table B.1 lists the State Farm driver identifiers for each partition. These were fixed once and reused without change across the baseline, cross-view and lighting stages, so no driver ever moved between partitions during the project.

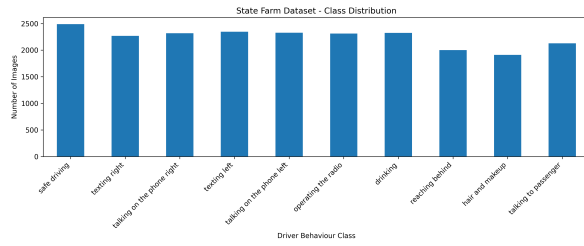
Table B.1: State Farm subject-wise split, by driver identifier.

Split	Images	Drivers	Driver identifiers
Train	15,899	18	p014, p015, p016, p021, p022, p024, p039, p042, p047, p049, p051, p052, p056, p061, p064, p066, p072, p081
Validation	3,439	4	p026, p035, p041, p050
Test	3,086	4	p002, p012, p045, p075

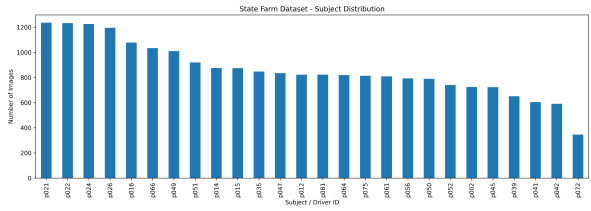
Table B.2: Composition of the unified multi-view metadata table (170,316 rows).

Dataset	View	Lighting	Train	Val	Test
State Farm	side	day	15,899	3,439	3,086
SAM-DD	front	indoor artificial	32,156	8,162	7,453
SAM-DD	side	indoor artificial	32,156	8,162	7,453
Low-Light DD	side	night	4,500	750	1,500
Low-Light DD	side	day	31,425	6,675	7,500
Cross-view experiment (State Farm + SAM-DD)			80,211	19,763	17,992
Lighting experiment (State Farm + Low-Light night)			20,399	4,189	4,586

The Low-Light DD daytime rows appear in the unified table but are left out of the lighting experiment on purpose, for the reason given in Section 3.2.3. Figures B.1a and B.1b show the State Farm class and per-driver image distributions behind these splits.



(a) Class distribution



(b) Images per driver

Figure B.1: State Farm dataset composition.

Appendix C

Full Per-Class Classification Reports

Supporting evidence for Sections 4.1, 4.2, 4.3 and 4.7. All figures are computed on the held-out subject-wise test partitions listed in Appendix B. The source CSV files are supplied with the code artefact under `results/reports/`.

Table C.1: Per-class F1 for the three single-view baseline models, State Farm test set.

Class	Baseline CNN	MobileNetV2	EfficientNet-B0	Support
c0 safe driving	0.620	0.707	0.738	316
c1 texting right	0.747	0.788	0.888	325
c2 phone right	0.928	0.970	0.966	338
c3 texting left	0.901	0.974	0.971	322
c4 phone left	0.944	0.917	0.880	345
c5 operating radio	0.783	0.960	0.900	327
c6 drinking	0.884	0.872	0.817	311
c7 reaching behind	0.698	0.935	0.878	293
c8 hair and makeup	0.619	0.708	0.600	251
c9 talking to passenger	0.403	0.572	0.520	258
Macro average	0.753	0.840	0.816	3,086
Weighted average	0.766	0.849	0.828	3,086

Table C.2: MobileNetV2 full per-class report, State Farm subject-wise test set (accuracy 84.67%).

Class	Precision	Recall	F1	Support
c0 safe driving	0.664	0.756	0.707	316
c1 texting right	0.960	0.668	0.788	325
c2 phone right	0.985	0.956	0.970	338
c3 texting left	0.972	0.975	0.974	322
c4 phone left	0.926	0.907	0.917	345
c5 operating radio	1.000	0.924	0.960	327
c6 drinking	0.819	0.932	0.872	311
c7 reaching behind	0.971	0.901	0.935	293
c8 hair and makeup	0.631	0.805	0.708	251
c9 talking to passenger	0.567	0.578	0.572	258
Macro average	0.849	0.840	0.840	3,086
Weighted average	0.861	0.847	0.849	3,086

Table C.3: Cross-view MobileNetV2, frozen-backbone against fine-tuned, mixed-view test set (17,992 images).

Class	Support	Frozen-backbone (75.94%)			Fine-tuned (91.76%)		
		P	R	F1	P	R	F1
safe_driving	9,006	0.887	0.881	0.884	0.944	0.977	0.960
texting_right	1,309	0.687	0.502	0.580	0.924	0.765	0.837
phone_right	1,332	0.722	0.642	0.680	0.840	0.986	0.907
texting_left	1,286	0.422	0.768	0.545	0.974	0.821	0.891
phone_left	1,341	0.591	0.782	0.673	0.906	0.899	0.903
adjusting_radio	327	0.928	0.746	0.827	0.974	0.927	0.950
drinking	1,273	0.754	0.610	0.675	0.840	0.833	0.837
reaching_behind	835	0.884	0.695	0.778	0.953	0.884	0.917
hair_or_makeup	1,025	0.696	0.537	0.606	0.867	0.876	0.871
talking_to_passenger	258	0.614	0.136	0.222	0.604	0.519	0.558
Macro average	17,992	0.719	0.630	0.647	0.883	0.849	0.863
Weighted average	17,992	0.781	0.759	0.761	0.918	0.918	0.916

Table C.4: Lighting stage: day-only baseline (night-only test set, 1,500 images), frozen-backbone and fine-tuned (merged day+night test set, 4,586 images). The baseline’s zero-recall rows are the collapse described in Section 4.3.

Class	Baseline (19.53%)		Frozen (62.30%)			Fine-tuned (77.91%)		
	P	R	P	R	F1	P	R	F1
safe_driving	0.000	0.000	0.559	0.712	0.626	0.551	0.845	0.667
texting_right	0.000	0.000	0.721	0.451	0.554	0.986	0.459	0.626
phone_right	0.000	0.000	0.689	0.598	0.640	0.900	0.883	0.891
texting_left	0.170	0.993	0.720	0.676	0.697	0.965	0.648	0.776
phone_left	0.000	0.000	0.780	0.523	0.626	0.926	0.653	0.765
adjusting_radio	0.000	0.000	0.732	0.851	0.787	0.998	0.960	0.979
drinking	0.000	0.000	0.392	0.764	0.518	0.638	0.889	0.743
reaching_behind	0.233	0.960	0.784	0.858	0.819	0.880	0.975	0.925
hair_or_makeup	0.000	0.000	0.423	0.414	0.419	0.596	0.663	0.628
talking_to_passenger	0.000	0.000	0.820	0.336	0.477	0.719	0.821	0.767
Macro average	0.040	0.195	0.662	0.618	0.616	0.816	0.780	0.777
Weighted average	0.040	0.195	0.664	0.623	0.621	0.822	0.779	0.779

Appendix D

Quantisation Raw Figures

Supporting evidence for Section 4.4 and Table 4.6. Exact file sizes were read directly from the filesystem. Megabyte figures are bytes divided by 1024^2 , matching the convention used in the project’s own results tables.

Table D.1: Exact model artefact sizes.

Artefact	Format	Bytes	MB
best_mobilenetv2_crossview_finetuned.keras	Keras float32	21,791,872	20.78
mobilenetv2_crossview_finetuned_int8.tflite	TFLite INT8	2,723,448	2.60
mobilenetv2_crossview_int8.tflite	TFLite INT8	2,721,672	2.60
mobilenetv2_crossview_dynamicrange.tflite	TFLite dynamic range	2,518,480	2.40
Compression ratio, float32 to INT8			8.00×

Table D.2: Quantisation before/after measurements. All samples are drawn from held-out subject-wise test partitions.

Model	Sample	n	Float32	INT8	Agreement
Cross-view frozen	mixed front+side	500	76.80%	76.00%	91.2%
Cross-view frozen	random side	300	59.30%	54.70%	—
Cross-view fine-tuned	random side	300	80.00%	77.30%	—
Cross-view fine-tuned	held-out sample	100	Keras vs. TFLite	100/100	
Cross-view fine-tuned	held-out sample	100	board vs. laptop	100/100	

The two board-versus-laptop agreement rows are the measurements that show the from-source ARM32 runtime is numerically faithful (Section 4.5.1). A dynamic-range variant was also produced and checked. It was not deployed, because the INT8 variant was the one carried through the accuracy evaluation.

Appendix E

FPGA Synthesis Report Excerpts

Supporting evidence for Section 4.6. Excerpts are taken word for word from `fpga/reports/utilization.rpt` and `fpga/reports/timing_summary.rpt`, produced by Vivado 2017.4 targeting 7a35tcp236-1, design state “Synthesized”, out-of-context.

Table E.1: Slice logic utilisation (from `utilization.rpt`).

Site type	Used	Available	Util.%
Slice LUTs	18	20,800	0.09
LUT as logic	18	20,800	0.09
LUT as memory	0	9,600	0.00
Slice registers	13	41,600	0.03
Register as flip-flop	13	41,600	0.03
Register as latch	0	41,600	0.00
F7 muxes	0	16,300	0.00
F8 muxes	0	8,150	0.00
Block RAM	0	—	0.00
DSP slices	0	—	0.00

All 13 registers are clock-enabled flip-flops with synchronous reset. Vivado’s own report carries the note that “the final LUT count, after physical optimizations and full implementation, is typically lower”, which is why Section 4.6 treats 18 LUTs as a conservative post-synthesis estimate rather than a placed-and-routed measurement.

Table E.2: Design timing summary (from `timing_summary.rpt`), clock `clk` constrained at 100 MHz.

Metric	Value	Failing endpoints
Worst negative slack (WNS)	+7.276 ns	0 of 17
Total negative slack (TNS)	0.000 ns	—
Worst hold slack (WHS)	+0.272 ns	0 of 17
Total hold slack (THS)	0.000 ns	—
Worst pulse width slack (WPWS)	+4.500 ns	0 of 13

The report concludes “All user specified timing constraints are met.” The estimated total on-chip power from `power.rpt` is 0.069 W (dynamic 0.000 W, device static 0.068 W) at a junction temperature of 25.3 °C, recorded by the tool at medium confidence. This is a vector-less estimate, since no real switching-activity file was supplied.

Appendix F

Alert Delivery Path and Its Verification

Supporting evidence for the alert-delivery pipeline summarised in Chapter 3. The board runs a continuous capture-classify loop whose hysteresis logic is a tick-for-tick Python copy of `distraction_alert_controller.v`. It uses the same 40% confidence threshold, the same eight-tick set and clear counters, and the same order of set and clear checks relative to counter increments, so an “uncertain” low-confidence tick holds both counters steady instead of resetting them, exactly like the RTL always-block does. On the rising edge of an alert, and only then, not once per classified frame, it posts the class, confidence and the JPEG-encoded triggering frame to a FastAPI backend. This backend stores the image and serves it to a Kotlin Android client, over the same bearer-token authentication used by the rest of the API.

A fully organic end-to-end trigger needs a person to hold a distracted pose in front of the camera for roughly seventy seconds, at this board’s inference speed. Because of that, verification was carried out in three independent layers instead.

Table F.1: Three-layer verification of the alert-delivery path.

Layer	What was exercised	Result
1. Logic equivalence	The Python hysteresis port replayed against the same five scenarios as the Verilog testbench (sustained safe, sub-threshold burst, sustained distraction, an uncertain tick mid-run, sustained safe again).	10/10 checks passed, matching the testbench’s expected transitions.
2. Hardware execution	Real webcam frames through the real INT8 model and the real hysteresis state machine, on the board, with no synthetic data.	Ran cleanly end to end at ~ 9.1 s per tick, no crash and no false alert.
3. Network delivery	The production posting function imported and called directly, then the stored record retrieved from the backend and viewed in the Android client.	HTTP 200; record and image confirmed present and byte-identical.

Layer 1 shows that the deployed decision logic behaves the same as the hardware design verified in Section 4.6. Layer 2 shows that the full pipeline runs on real hardware. Layer 3 shows that the network call the loop actually makes reaches its destination. What the three layers

together do not add up to is one single unattended physical demonstration. That limitation is stated here directly, not hidden.

Appendix G

Physical Buzzer Alert Output

Supporting evidence for the buzzer paragraph in Chapter 3. This appendix covers why the board could not drive a buzzer at all before this work, what was built to fix that, the real problems hit along the way, and how the buzzer itself was diagnosed and driven correctly.

G.1 The board could not control the header pin at all

The available buzzer is a real two-pin passive unit, with no transistor or driver circuit to hand, so it had to be wired directly to a GPIO-capable header pin. Checking `/sys/class/gpio` on the board's stock PYNQ image showed only one exposed GPIO controller, `zynq_gpio` (118 lines, 54 processor MIO pins plus 64 EMIO pins), and none of those lines reach the board's PMOD headers. The PMOD and LED-adjacent pins sit on FPGA-fabric I/O banks, not the processor's fixed MIO banks. No bitstream or overlay file exposing them existed anywhere on the board's filesystem. Nothing on this board could toggle a header pin from Python without a custom bitstream built specifically to expose it. This was worked out by direct inspection, not assumed.

G.2 The overlay

A minimal Vivado project was built from scratch at `fpga/vivado_overlay_buzzer/`, targeting `xc7z020c1g484-1`, the real device on the board (distinct from the `distraction_alert` project's Basys3 target used in Section 3.5). The block design (Figure G.1) consists of a Zynq7 Processing System block and a single-bit AXI GPIO block. The processing-system block does not reconfigure the processor: Linux is already running from the SD card independently of this design. It is included so Vivado generates the AXI clock, reset and address-decode plumbing that lets the already-running processor address a peripheral built in the reconfigurable fabric. The GPIO block's output was brought out to package pin V7, which the board's user guide identifies as header J62 pin 1 (PMOD2_0, LVCMOS33) (Xilinx, 2019).

Three real build errors were hit and worked through by reading Vivado's own output rather than guessed in advance. A manual clock-connection automation call errored because the AXI4 connection automation had already wired the clock. The redundant call was removed. `make_bd_pins_external` does not accept a `-name` option in this Vivado version, confirmed against its own `-help` text, so the external port was renamed after creation instead. The generated wrapper Verilog file was assumed to land under a `.gen` build-products directory and was in fact under `.srcs`, so the add-files step had to be corrected. Synthesis, implementation

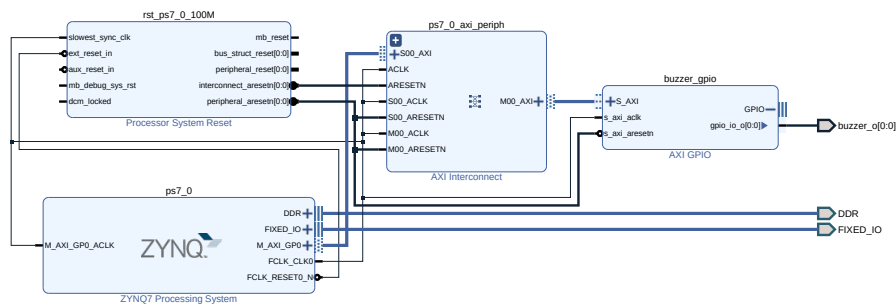


Figure G.1: The buzzer overlay’s Vivado block design. A Zynq7 Processing System block provides AXI/clock/reset plumbing to the already-running processor. A single-bit AXI GPIO block is the only new circuit, exposed externally as `buzzer_o` on package pin V7.

and bitstream generation then finished with zero errors. `write_hwdef` produced a `.hwdef` file that turned out to already be a ZIP archive containing a valid `.hwh` hardware-handoff file. This is the metadata PYNQ needs to know the GPIO block’s address on the AXI bus, alongside the bitstream itself, which configures the fabric electrically but carries no address information of its own. No separate `.hdf-to-.hwh` conversion step was needed for this Vivado version.

G.3 Root permission and two unrelated Jupyter defects

Loading the overlay (`pynq.Overlay(...)`) initially failed with `RuntimeError: Root permission needed by the library`. This was traced to a hardcoded check in `pynq/xlnk.py` (`if os.getuid() != 0: raise RuntimeError(...)`), not a file-permission problem. Giving the `xilinx` user read and write access to `/dev/xlnk` through `udev` rules was tried first, and testing confirmed it did not fix it. The fix that actually worked was changing the `jupyter.service` systemd unit from `User=xilinx` to `User=root` and restarting the service. This is the standard PYNQ convention on Xilinx’s own reference image. This particular community board image had simply been set up differently. A live kernel process owned by `root` after the change confirmed the fix worked. Two unrelated defects in the same service were found and fixed while debugging this. `HOME` was wrongly set to `/root` for every kernel the service spawned, which broke the `tflite_runtime` import used throughout the on-device work (Section 3.4.2), since it lives under `/home/xilinx`. The configured notebook directory had also drifted away from where the board’s own notebooks actually live, hiding them from the browser’s file tree. Both were fixed in the service’s own startup script.

G.4 Driving a passive buzzer

Holding the GPIO pin at a steady logic level produced a single click on each transition, not a tone. This is the expected behaviour of a “passive” buzzer, which has no built-in oscillator and only responds to a changing voltage, unlike an active buzzer that would tone continuously under steady DC. The fix was to generate an actual square wave in software. This used a tight busy-wait loop polling the clock rather than `time.sleep()`, which produced a noticeably less jittery tone, at the cost of pinning a CPU core for the whole alert. A frequency sweep from 300 Hz to 4,000 Hz was used to pick 2,500 Hz by ear as the clearest and loudest. Some residual timing instability is inherent to generating a waveform in software on a non-real-time operating system, and this was accepted rather than chased further, since it does not affect whether the alert is actually audible.

G.5 Integration

`alert_loop_infer.py` lazily loads the overlay once per process and caches the GPIO pin handle, since loading an overlay is too slow to repeat on every alert. The buzzer is sounded on the same rising edge as the backend post, inside the same alert-handling branch, so the two channels never diverge. The fix was confirmed working through the real Jupyter server rather than a directly spawned kernel, since the server’s own process, not a manually launched one, is what carries the corrected root permission.

Appendix H

Repository and Notebook Map

The full source code, notebooks, RTL and supporting documentation are submitted separately as a plain-text artefact file, as required by the FPR submission rules. This appendix maps that artefact to the project’s internal structure.

Table H.1: Notebook map: AI pipeline.

File	Purpose
notebooks/01-05	Dataset exploration, subject-wise split, baseline CNN, transfer-learning models, evaluation (Section 4.1).
notebooks/06	Lighting-domain robustness: day-only baseline, frozen-backbone and fine-tuned training on real night data (Section 4.3).
notebooks/07	Multi-view dataset review (no training).
notebooks/08	Unified multi-view dataset builder; adapters for State Farm, SAM-DD and the Low-Light dataset.
notebooks/09	Cross-view MobileNetV2 training, both phases (Section 4.2).
notebooks/10	Synthetic front-view augmentation plan (planning only; no synthetic images were generated or used anywhere in this project).

Table H.2: FPGA, embedded deployment and alert-delivery map.

File	Purpose
fpga/rtl/*.v	Alert controller, UART receiver and top-level Verilog RTL (Sections 3.5, 4.6).
fpga/sim/*_tb.v	Self-checking testbench and real-model-data testbench.
fpga/scripts/run_synth.tcl	Out-of-context Vivado synthesis script producing the reports in Appendix E.
fpga/notebooks/01-04	Runnable notebooks reproducing RTL simulation, synthesis, TFLite conversion and live board checks.
fpga/vivado_overlay_buzzer/	Vivado project building the PYNQ-loadable overlay that exposes the buzzer GPIO pin (Appendix G).
fpga/board_scripts/	On-device inference scripts: single-shot webcam, HDMI capture, and the continuous alert loop with hysteresis and buzzer output.
fpga/board_notebooks/	Board-hosted demonstration notebooks (live camera, HDMI capture, video file).
mobile_app/backend/	FastAPI alert backend receiving alert events and images from the board.
mobile_app/DistractioAlertApp/	Kotlin Android client displaying alert history and captured frames.

Appendix I

Supporting Documentation

The following documents were kept up throughout the project alongside the code and give detail beyond what is reported in the main body. They are supplied with the code artefact.

- `docs/PROJECT_STATUS.md` - current-state summary maintained across the whole project.
- `docs/multiview_stage_progress_log.md` - chronological log of the multi-view and dataset stages.
- `fpga/zc702_board_bringup_log.md` - detailed log of the ZC702 bring-up, including the full ARM32 runtime diagnosis (Section 3.4.2), the preprocessing-mismatch investigation (Section 4.5.1), and the WiFi hypothesis test described in Section 5.4.
- `docs/unified_multiview_dataset_format_plan.md` - the unified metadata schema definition (Section 3.2).
- `docs/multiview_driver_distraction_dataset_review.md` - the candidate dataset review that informed the choice of SAM-DD.
- `docs/survey/` - the systematic survey underpinning Chapter 2, including its full reference list and PRISMA figures.
- `fpga/README.md` - scope statement for the FPGA alert-controller work, stating explicitly what is verified and what is not claimed.